HAMK
Häme University
of Applied Sciences

Azure template

Bachelor’s thesis
Degree Programme in Computer Applications
Spring 2024

Aleksandr Sohhatov

HAMK
Häme University
of Applied Sciences

Degree Programme in Computer Applications
Author Aleksandr Sohhatov
Subject Azure template
Supervisors Ismo Turve

Abstract
Year 2024

The purpose of the thesis was to explore Infrastructure as Code as a revolutionary solution for automating and provisioning infrastructure on cloud computing platforms, with a particular focus on Microsoft Azure. This study addressed the challenges associated with manual infrastructure provisioning, emphasizing the need for agility, efficiency, and scalability in contemporary business operations. The research questions centered on understanding how Infrastructure as Code streamlines and automates Azure infrastructure provisioning, identifying the perceived benefits and challenges of adopting Infrastructure as Code among IT professionals, and examining the impact of Infrastructure as Code on technology acceptance in the context of Azure infrastructure provisioning.

The thesis adopted a practical approach, incorporating theoretical insights into technology acceptance and cloud infrastructure management. It commenced with exploring fundamental concepts related to cloud computing, manual infrastructure provisioning challenges, and the principles of Infrastructure as Code. The research methods included a literature review, field experiments, and demand analysis, aiming to provide comprehensive insights into the effectiveness and acceptance of Infrastructure as Code in Azure infrastructure provisioning.

The findings indicated that Infrastructure as Code significantly improves the efficiency, consistency, and scalability of Azure infrastructure provisioning compared to manual approaches. IT professionals perceived Infrastructure as Code as beneficial in simplifying scripting tasks, enhancing agility, and enabling seamless integration with version control systems like Git. However, challenges like the learning curve associated with Infrastructure as Code tools and compatibility issues with existing infrastructure were identified.

Based on the analysis, it is recommended that organizations invest in training programs to familiarize IT professionals with infrastructure, such as code tools and methodologies. Additionally, continuous monitoring and feedback mechanisms should be established to address challenges and optimize the implementation of Infrastructure as Code in Azure environments.

Keywords Infrastructure as Code, Continuous Integration/Continuous Deployment, Domain-Specific Language.
Pages 65 pages

Glossary

Cloud Computing: Utilizing remote servers over the internet to store, manage, and process data.

Infrastructure as Code (IaC): Defining and managing infrastructure through machine-readable files.

Microsoft Azure: A cloud computing service by Microsoft for building, deploying, and managing applications.

Manual Provisioning: Setting up resources individually without automation.

YAML: Human-readable data serialization standard, often used in configuration files.

JSON (JavaScript Object Notation): Lightweight data interchange format, often used in configuration files.

Azure Resource Manager: Azure service for managing resources and automation.

Ansible, Chef, Puppet: Third-party tools for automating infrastructure management.

Declarative Nature: Specifying desired outcomes rather than the steps to achieve them.

Continuous Integration/Continuous Deployment (CI/CD): Automating software development processes.

Bicep: Domain-specific language for defining Azure resource configurations.

HashiCorp Terraform: Tool for building, changing, and versioning infrastructure.

Ease of Use: User-friendliness and simplicity.

Version Control: Managing changes to software code or configuration files.

Perceived Usefulness: Subjective assessment of how a technology improves performance.

Technology Acceptance Model (TAM): Predicts technology adoption based on perceived ease and usefulness.

Unified Theory of Acceptance and Use of Technology (UTAUT): Predicts technology adoption comprehensively with additional factors.

Declarative vs. Imperative IaC: Describing desired state vs. specifying actions for infrastructure.

Idempotency: Applying same configuration produces same result, reducing unintended changes.

Domain-Specific Language (DSL): Specialized language for specific domain tasks.

Content

1 Introduction 1

2 Theories and Literature Review 3

2.1 Technology Acceptance Models 3

2.2 Relevance of Technology Acceptance Theories in Infrastructure as Code Adoption 4

2.3 Infrastructure as Code: Concepts and Principles 4

2.4 Benefits of Infrastructure as Code 6

2.5 Challenges and Limitations of Infrastructure as Code 6

2.6 Microsoft Azure and Its Typical Deployment Approach 7

2.6.1 Microsoft Azure Advantages 8

2.6.2 Limitations of Microsoft Azure 9

2.6.3 Azure DevOps 10

2.6.4 Azure Pipeline 12

2.6.5 Deploying Azure Resources with Bicep 13

3 Methodology 17

3.1 Research Design 17

3.2 Sampling, Data, and Data Collection 18

3.3 Analysis 19

3.4 Ethical Considerations 19

3.5 Study Limitations 19

4 Project Implementation 20

4.1 Project Design 20

4.2 Bicep Templates 20

4.3 Pipeline 25

4.4 Azure DevOps 28

5 Results 39

5.1 Descriptive and Inferential Statistical Analysis 40

5.1.1 Effect of Infrastructure as a Code on Deployment Time (minutes) 43

5.1.2 Effect of Infrastructure as a Code on Deployment Perceived Ease of Use (1-10) 44

5.1.3 Effect of Infrastructure as a Code on Deployment Error Rate (%) 45

5.1.4 Effect of Infrastructure as a Code on Deployment Consistency Rate (%) 45
5.1.5 Effect of Infrastructure as a Code on Deployment Task Success Rate (%) 46
5.2 Discussion on the Infrastructure as Code and Its Features 47
6 Summary 49
7 References 51
Appendix 1: Material Management Plan 1
Appendix 2: Survey and Interview Questions 2

Figures, program codes, commands, and tables

Table 1: Azure manual deployment limitations 10

Table 2: Azure DevOps options for developers 11

Table 3: Azure DevOps services and descriptions 12

Table 4: Main features of bicep 14

Table 5:Bicep, JSOn, and HashiCorp (HCL) syntax 15

Table 6: Fieldwork dataset 39

Table 7: Crosstabulation of experience and Infrastructure as Code preferences in cloud infrastructure deployment 41

Table 8: Descriptive statistics 42

Figure 1: Program code 1 highlighting project tree 21
Figure 2: Program code 2 highlighting project resource description 21

Figure 3: Program code 3 highlighting setting décor parameters and global unique identifier 21
Figure 4: Program code 4 highlighting securing parameters 22
Figure 5: Program code 5 highlighting getting, storing, and retrieving the value of the sasToken 22
Figure 6: Program code 6 highlighting the name, local, kind, and SKU declaration, and Storage Account dependencies specification 23
Figure 7: Program code 7 highlighting BlobStorages declaration using bicep loop 23
Figure 8: Program code 8 highlighting creating a blobStorage for each containerNames 23
Figure 9: Program code 9 highlighting services deployment to secure cloud application 24
Figure 10: Project code 10 highlighting the creation of a secret to store the sasToken 24
Figure 11: Program code 11 highlighting the parameter JSON scrip used in deploying the Azure Key Vault 25
Figure 12: Program code 12 highlighting trigger and bicep linter implementation script 26
Figure 13: Program code 13 highlighting bicepconfig.json reconfiguration 26
Figure 14: Program code 14 highlighting bicep code validation 27
Figure 15: Program code 15 highlighting the deployment script 27
Figure 16: Program code 16 highlighting script that publishes built artifact to Azure pipeline 27
Figure 17: The program Infrastructure as a Code stages 28
Figure 18: List of jobs 28
Figure 19: Artifacts 29
Figure 20: Pipeline library used to store the values and secrets 29
Figure 21: Release Pipeline 30
Figure 22: Output generated after successfully running the pipeline scripts 30
Figure 23: Successful deployment details 31
Figure 24: Program code 17 highlighting the three created parameters 31
Figure 25: Program code 18 highlighting workflow deployment 31
Figure 26: Logic app 32
Figure 27: Program, code 19 highlighting trigger type and settings variables 32
Figure 28: Recurrence frequency 32
Figure 29: Program code 29 showing action type definition 33
Figure 30: Program code 21 highlighting API connections declaration 33
Figure 31: API connections associated with the logical app output 34
Figure 32: Program code 22 highlighting actions blocks definitions 34
Figure 33: List of blobs 34
Figure 34: Program code 23 showing the control structure that compares the data in the workflow 35

Figure 35: Program code 24 highlighting condition expression in the specified expression .. 35
Figure 36: Getting blob 36
Figure 37: Program code 25 Action type ApiConnection parameters 36
Figure 38: Program code 26 highlighting fragmented messages configuration 36
Figure 39: Program code 27 used to delete the blob 37
Figure 40: Delete blob in Azure logic 37
Figure 41: Program code 28 highlighting the script used to implement the “Set Variable” action 37
Figure 42: Interface for setting variable 38
Figure 43: Program code 29 highlighting for initializing variable 38
Figure 44: Initializing variable 38
Figure 45: Comparison of experience in cloud computing and preference for Infrastructure as a Code 41
Figure 46: Deployment Time T-test 43
Figure 47: Perceived Ease of Use t-test 44
Figure 48: Error Rate t-test 45
Figure 49: Consistency t-test 46
Figure 50: Success Rate t-test 47

1 Introduction

According to Deb and Choudhury (2021), businesses are increasingly adopting cloud computing across many platforms, including but not limited to AWS, Microsoft Azure, Google Cloud Platform, Oracle Cloud, IBM Cloud, and Alibaba Cloud, to name a few, host applications and services. Nevertheless, getting started with such platforms poses challenges, especially for ordinary users who do not know how to set them up (Chieu et al., 2010). Manual configuration and deployment to cloud computing platforms come with several challenges.

In the absence of an automated approach, the manual process is time-consuming as manually creating virtual machines, app services, and storage accounts takes valuable time and resources despite being repeated tasks that can be automated (Stefanovic and Katinski, 2021. Besides, manual configurations are error-prone, especially considering repeated tasks, which result in burnout that often results in IT professionals making more such errors. According to Konala et al. (2023), overlooking settings or misconfigurations are inevitable with manual infrastructure provisioning, which may result in more adverse outcomes, including but not limited to security vulnerabilities, deployment failures, and operational inefficiencies. Additionally, manual configuration falls short of consistency across environments. Scalability concerns arise with manual application provisioning.

Several solutions exist that attempt to address manual deployments, but not without limitations. Considering this study focuses on Microsoft Azure, most of the solutions presented are related. Azure Resource Manager templates allow users to use YAML syntax or JSON to define Azure resource configurations and infrastructure (Patni et al., 2020). Organizations have adopted third-party configurations as an approach to solving the limitations of manual infrastructure provisioning. For instance, Hill (2021) outlines tools including but not limited to Ansible, Chef, and Puppet, which enterprises currently adopt to manage Azure infrastructure provisioning. It is worth acknowledging that these third-party tools offer establishments powerful Azure infrastructure configurations and automation. However, they, too, have limitations associated with necessitating high-level expertise to not only set up and maintain. Besides, these third-party tools pose incompatibility issues.

This study proposes Infrastructure as Code as the ultimate solution to automating and provisioning infrastructure on cloud computing platforms. According to Morris (2016), this involves defining configurations as code, allowing IT professionals and cloud computing users to automate deployments.

Following are the project's research questions:

I. How does Infrastructure as Code streamline and automate Azure infrastructure provisioning compared to manual approaches?

II. What are the perceived benefits and challenges of adopting Infrastructure as Code for Azure infrastructure provisioning among IT professionals?

III. What is the impact of Infrastructure as Code on technology acceptance in the context of Azure infrastructure provisioning?

2 Theories and Literature Review

The literature review focuses on the effect of Infrastructure as Code on the comfortability of deployments and infrastructure provisioning, especially in cloud system platforms. It examines existing studies on adopting Infrastructure as Code and its influences on technology acceptance. It draws upon theories: the Technology Acceptance Model (TAM) and the Unified Theory of Acceptance and Use of Technology (UTAUT). The review highlights how Infrastructure as Code enables faster operations and more considerable scalability and, of course, helps to deploy faster while considering the issues and concerns connected with its adoption.

2.1 Technology Acceptance Models

According to Alshammari et al. (2020), technology acceptance studies, for instance, those covering TAM and UTAUT, have proven reliable in understanding the adoption of Information as a Code in cloud computing. Such theories offer an overview of how people think and act in devising and utilizing technology, which is very important for rapidly adopting Information as a Code since it is a powerful tool that revolutionizes cloud infrastructure management processes.

Davis introduced TAM in 1989, and the relationship between perceived usefulness and perceived ease of technology usage was used to determine the user's intention to adopt a particular technology. Alshammari et al.(2020) argue that the conclusion can be presented in the form that correcting a false belief will result in a higher probability of a person adopting a technology that makes his or her performance better and easy to use. In adopting Information as a Code, TAM can aid in unveiling what promotes cloud engineers and DevOps teams to utilize Infrastructure as a Code (Chijioke-Uche, 2022). Herein, for example, cloud engineers could be aware and realize that if Information as a Code framework, like Terraform or AWS CloudFormation, makes it easier for provisioning and managing the cloud resources, they are more likely to accept such tools compared to others since they sense that these frameworks make their job more straightforward and more efficient.

UTAUT2 provides a broader, in-depth perspective on technology adoption by adding other critical constructs from different models. As Chijioke-Uche (2022) suggests, this theory focuses on performance expectancy, effort expectancy, social influence, and facilitating conditions in accounting for individual intentions to use technology. Besides, Ljunggren

(2023) adds that UTAUT2 also considers these additional categories, i.e., hedonic motivation, cost/perceived value, and habit, that make consumer technology adoption more relevant. In studying Infrastructure as a code adoption in cloud computers, UTAUT2 might offer information concerning individual users and explain why they adopt information as a code practice. UTAUT2 is the framework for understanding two parties: a user and an information system and technology that helps explain an innovation's adoption or rejection. For illustration, Gutta (2023) writes that if developers understand that the Infrastructure as a Code provides fast deployment speed with fewer errors, they are more likely to integrate the Infrastructure as a Code into their process.

2.2 Relevance of Technology Acceptance Theories in Infrastructure as Code Adoption

Guerriero et al. (2019) and Hasan and Ansary (2023) developed fine-detailed literature around adopting Infrastructure as Code in a cloud computing context. The research focused on the theories' most significant findings and examples, including the TAM and UTAUT. Guerriero et al. (2019) argue for the need for agility infrastructure management and explain how Infrastructure as Code simplifies processes, which eases the deployment; these fit in with TAM factors such as usefulness and ease of use as they guide the use of technology. The supporters claim that the infrastructure of the Infrastructure as Code facilities for the customer is streamlined, which, in turn, enhances efficiency and scalability, and both equally affect the acceptance of the technology. Besides this, Hasan and Ansary's (2023) studies also point to the agile behavior of Infrastructure as Code and the laws of load balancing in environments while stressing the performance expectancy and the effort expectancy emphasis stressed by UTAUT. They assert that Infrastructure as Code’s agility accommodates fluctuating capacity demands and improves infrastructure configurations, which aligns with the user's high-tech savviness and the expectation that tech should be practical and friendly.

2.3 Infrastructure as Code: Concepts and Principles

Infrastructure as a Code is changing the usual approach to provisioning and managing infrastructure resources based on treating infrastructure configurations as software records. Infrastructure as a Code, described by Gutta (2023), includes an implementation where infrastructure needs are articulated in a code-based representation, be it scripts or configuration files, instead of manual tools. Sandobalin et al. (2020) stipulate that this strategy offers a mechanism to execute the commands through str, producing a consistent

machine interface, stabilizing enhanced liability, improving scalability, and reducing human errors and configuration drift. One such example is a firm like Netflix, which uses Infrastructure as a Code and institutes software tools to configure infrastructures in the cloud and, thus, quickly and reliably deploys streaming services glacially across different regions with small manual input.

The fundamental principles and concepts underlying Infrastructure as a Code encompass declarative vs. imperative approaches, idempotency, and version control. According to Code (2020), an Infrastructure as a Code in a declarative format describes the desired state of the infrastructure without showing the exact sequence of actions to be carried out for that desired state. Compared to the declarative Infrastructure as a Code discussed by Hasan and Ansary (2023), the type of Infrastructure as a Code that closely regulates provision and infrastructure configuration is called imperative Infrastructure as a Code. Take, for instance, when following a declarative system, a configuration file might have been written stating that a specific number of virtual machines must run at all times, but when following a thorough script, a program would have to specify precisely how to create and configure each virtual machine individually.

The concept of idempotency is one of the main ideas of Alonso et al. (2022), who state that the code of infrastructure does not depend on whether it is applied several times. The result always remains the same, regardless of the original state. This reduces the chance that the infrastructure configurations are altered due to the dynamic cloud environment. Patni et al. (2020) also show the link to this system. He/she says that if an Infrastructure as a Code script specifies that a particular software package is to be installed on a virtual machine, then repeated running of this script will not lead to duplication of this software. Not the next software installation and unintentional changes in the system configuration this time. Version control, reported by Kaledio and Lucas (2024), allows keeping traces of changes to infrastructure code in time, permitting distributed work, verification, and rollbacks. With these sorts of version control repositories such as Git, the code of infrastructure can be stored, so the team can easily manage the changes, review the code modifications, and restore the code to the previous versions if required. While dealing with different models, the start-up development team will use Git to keep track of their infrastructure as a code template provision. This will help the team to make it easy to include their teammates.

2.4 Benefits of Infrastructure as Code

Infrastructure as a Code centers on ease of use and usefulness, which are the TAM core components. Based on the research by Kumara et al. (2021), Infrastructure as a Code facilitates the consolidation of complex operations by translating human commands into machine-readable Code and adds to the subjective ease of use principle. Infrastructure automation imparts this ability to users with no coding expertise to provision and deploy infrastructure, which is even more advantageous for them. In addition to this, authors like Gutta et al. (2023), Ljunggren Citing (2023), and Kumara et al. (2021) remark on three things that make Infrastructure as a Code extremely useful: resource utilization, scalability, and agility, which enable quick deployment and rapid feature scaling. All these benefits help convince organizations that IAC is billed as a job performance-supporting tool; individuals see them as valuable in job performance increases.

Additionally, agility and responsiveness are vital benefits channeled via the Infrastructure as a Code methodology. Gutta (2023) states that Infrastructure as a Code makes the organization adaptable to the changed business needs by automating infrastructure deployments and configurations. They create this agility, like Spotify, where managers give the Infrastructure as a Code tool the power to roll out new features or update a couple of times a day, allowing them to be first on the market and react quickly to any changes. Furthermore, Achar(2021) pointed out that the Infrastructure as a Code tool constitutes the base for devotion to rapid iterations and experiments; it is possible to realize innovative ideas and their transformations quickly, however, without ruining the trust data. For instance, companies such as Google utilize Infrastructure as a Code to employ new motivation mechanisms and enable them to evolve their systems at any given time repeatedly.

2.5 Challenges and Limitations of Infrastructure as Code

The Infrastructure as Code uptake is a complicated process governed by several attributed impacting elements, such as the complexity of the technology itself, how easy it is to adopt Infrastructure as Code, and organizational and cultural factors. The application of theories for technological acceptance, such as TAM and UTAUT, can expose the effects of microservice adoption through a lens of users' perceptions and the organization's readiness level.

Patni et al. (2020) point out that the main reasons this transition process is perceived as complex and demanding a steep learning curve are the learning curve and the complexity that must be taken on. According to TAM, the perceived complexities of technology usage

inhibit people's willingness to use technology. The attitude of individuals with no experience coding towards the native tools for Infrastructure as Code might be viewed as complicated and an obstacle to studying, thus causing a loss of excitement to practice it. Kaledio and Lucas (2024) have remarked that in severing ease of use, a vital variable of TAM follows on to inspire users' positive opinions about Infrastructure as Code. Organizations with diverse skill sets may have some teams adopt Infrastructure as Code faster than others; for example, teams with more members who are more familiar with coding will be more eager to embrace Infrastructure as Code than their counterparts who are less conversant with the same. Based on the works of Pessa (2023), the adoption and implementation of Infrastructure as Code tools might be affected by how valuable people think they are and their pre-conceptions.

Organizations use various tools and approaches to carcass selection based on compatibility with the current systems, plug-and-play nature, and the potential to increase productivity and enhance efficiency. Moreover, regarding the intricacy of tooling and ecosystems, Pessa (2023) points towards giving users the necessary training and support since this is a way to overcome problems that could appear in the context of perceived behavioral control and self-efficacy. One of the cultural factors of employees found by Patni et al. (2022) is the fear of job displacement and unwillingness to adopt new technologies. Overcoming such fears and resistance to new technologies necessitates persuasive communication and organizational change strategies that are directed by theories of technology acceptance and adoption (TAM and UTAUT). The more organizations focus on Infrastructure as Code as an enabler of operations streamlining, scalability improvement, and faster deployments, the more likely they are to create the environment for the knowledge of users willing to accept the technologies.

2.6 Microsoft Azure and Its Typical Deployment Approach

Azure Cloud Platform, developed by Microsoft, is a comprehensive and powerful cloud computing platform that provides businesses and individuals with a wide range of services and tools. According to Jennings (2010), with Azure, organizations can leverage the benefits of cloud computing, such as scalability, flexibility, and cost-efficiency, to drive innovation, enhance productivity, and meet their unique needs. Microsoft has established an extensive network of data centers worldwide, allowing users to deploy their applications and services closer to their customers. This not only reduces latency but also ensures high availability and resilience. Azure's global presence and network connectivity options enable businesses to expand their reach and confidently deliver their services globally (Jennings, 2010).

Security and compliance are paramount in the Azure Cloud Platform. Microsoft has implemented robust security measures to protect data and applications. Rath et al. (2019) affirm that Azure offers rest and transit encryption, multi-factor authentication, and advanced threat detection systems. Compliance certifications from various regulatory bodies ensure that Azure meets the strictest security and privacy standards, making it suitable for organizations operating in highly regulated industries. Microsoft Azure is a one-stop platform with over 200 products and cloud services (Khan & Chandaka, 2021). Today, the platform supports innovative solutions in various public life areas- politics, healthcare, finance, manufacturing, and retail. Many opportunities are presented for the Internet of Things, data science, and other technologies, without which it is impossible to imagine modern scientific development. At the same time, MS Azure is used by many Fortune 500 companies, including Walmart, FedEx, and HP®.

2.6.1 Microsoft Azure Advantages

Azure has a wide set of capabilities compared to competitors, covering the entire range of corporate IT tasks. MS Azure offers clients a flexible cooperation scheme, with the main forms being: Microsoft Azure's Platform as a Service (PaaS) offers a complete development and deployment environment in the cloud with resources that allow users to deliver everything from simple cloud applications to complex enterprise cloud applications (Yasrab, 2018). They buy the resources they need following a pay-as-you-go pricing, resulting in users paying for only those services they use. Microsoft Azure's SaaS (software as a service) allows users to connect to and use cloud-based apps over the Internet (Padhy et al., 2011). This also includes providing network access to software applications (Salesforce CRM, Microsoft Office 365, and others). Microsoft Azure's Infrastructure as a Service (IaaS) is a cloud computing service offering the required computing, storage, and networking resources on demand (Shahzadi et al., 2017).

Besides, Microsoft Azure allows the implementation of convenient services that work with databases and big data. Azure offers customers relational databases (SQL Azure) and NoSQL databases (Cosmos DB), as well as servers to support the most popular and reliable products from other companies (MySQL, Maria DB, PostgreSQL, Redice) (Seara et al., 2021). The Azure Data Explorer cloud platform provides many opportunities for analyzing and exploring big data. Additionally, Microsoft Azure allows the implementation of multiplatform tools in the development field. Azure allows users to deploy open-source solutions built with .NET, Java, JavaScript (NodeJS), PHP, Python, and Go (Soni, 2020). The same goes for operating systems (including Linux), which Azure allows users to deploy to the

cloud. Besides, Microsoft Azure's support development environments include popular ones such as VS Code, Visual Studio, IntelliJ, and Eclipse.

Moreover, security is one of Microsoft products' basic competitive advantages. The company invests more than $1 billion annually in cybersecurity R&D. Azure provides customers with extensive application security management capabilities. To protect user data, Microsoft Azure uses technologies such as Microsoft Sentinel (a scalable cloud-centric solution for managing information security) and Microsoft Defender (a service that contains the tools necessary to protect resources, monitor the state of security, protect against cyber-attacks, and optimize security management) (Loaiza Enriquez, 2021).

2.6.2 Limitations of Microsoft Azure

Overall, it takes a longer lead time to deploy Applications on Azure securely, especially among beginners. The speed of service is determined by the ability to quickly create, test, deploy, and scale new applications. According to Familiar (2015), it was also worth noting that solutions built with Azure can take longer to get to market due to the availability of many out-of-the-box tools, such as templates and documentation, that are often difficult to comprehend and master. Besides, despite the orientation of Microsoft Azure technologies providing support on how to use the portal, Azure primarily avails traditional approaches to provisioning infrastructure on the platform, which has shortcomings as follows.

Azure manual deployment limitations highlight challenges including time-consuming processes, error-prone configurations, lack of consistency, scalability limitations, security vulnerabilities, and inefficient resource management are presented in

Table 1.

Table 1: Azure manual deployment limitations

Challenge	Description
Time-consuming process	Manually creating virtual machines, app services, and storage accounts involves navigating through the Azure portal and configuring each resource individually, which consumes valuable time and resources, especially for repeated tasks.
Error-prone configurations	Manual configurations are prone to errors, especially when considering repetitive tasks. This leads to burnout among IT professionals and results in deployment failures, security vulnerabilities, operational inefficiencies, and costly downtime.
Lack of consistency across environments	Manual deployments lead to discrepancies between development, staging, and production environments, resulting in issues related to inconsistencies in security, performance, and configurations. This necessitates troubleshooting efforts and potential downtime.
Scalability limitations	Manual cloud configurations limit scalability, making it challenging to scale resources up or down on demand, resulting in inefficient, error-prone, and cumbersome scaling processes that hinder responsiveness and agility, especially in dynamic environments.
Security vulnerabilities and compliance violations	Manual configurations may lead to misconfigurations or overlooked settings, exposing deployed virtual machines to unauthorized access or sensitive data exposure, potentially resulting in compliance violations, data loss, and the need for extensive troubleshooting and remediation efforts.
Inefficient resource management	Manual interventions are required for provisioning, scaling, and de-provisioning resources. This hinders efficient resource management and may lead to underutilization or over-provisioning, which impacts cost-effectiveness and operational efficiency.

2.6.3 Azure DevOps

Azure DevOps is a group of Microsoft Azure services that allow collaboration on a project from planning to deployment (Soni, 2020). In addition to the general advantages of Azure (speed, scalability, security), Azure DevOps provides developers with many additional options, including:

Table 2 presents Azure DevOps options for developers outline working in Azure cloud and locally, connecting through web browsers and IDEs, integrating with popular services and GitHub, writing custom extensions, and using Azure Pipelines for automated project creation and testing.

Table 2: Azure DevOps options for developers

Option	**Description**
Work in Azure cloud and locally using Azure DevOps Server	Developers can work in the Azure cloud and locally by utilizing Azure DevOps Server, which combines a version control system, data collection, and reporting on a local server, offering flexibility in development environments.
Connect to DevOps functions through a web browser and IDE	Azure DevOps allows developers to connect to DevOps functions through a web browser and an Integrated Development Environment (IDE), providing seamless access to project management, version control, build and release pipelines, and other DevOps tools regardless of the preferred interface.
Integration with popular services and GitHub repository	Azure DevOps offers integration with various popular services such as Campfire, Trello, UserVoice, and the GitHub repository. This enables teams to streamline workflows and collaborate more effectively by leveraging familiar tools and platforms within the DevOps ecosystem.
Write custom extensions using standard web development tools	Developers can create custom extensions for Azure DevOps using standard web development tools like JavaScript (JS), HTML, and CSS, empowering them to extend the functionality of Azure DevOps to suit their specific requirements and integrate additional features seamlessly into their DevOps processes.
Automatically create and test projects using Azure Pipelines service	Azure DevOps provides the Azure Pipelines service, which enables the automatic creation and testing of projects, facilitating Continuous Integration (CI) and Continuous Deployment (CD) workflows. With Azure Pipelines, teams can automate the build, test, and deployment processes, ensuring efficient and reliable software delivery pipelines.

The table below shows some of the Microsoft Azure DevOps services that were particularly important in this study.

Table 3 presents Azure DevOps services include Azure Artifacts for package sharing, Azure Boards for project planning and management, Azure Pipelines for automated project creation and testing, Azure Repos for version control, and Azure Test Plans for comprehensive application testing.

Table 3: Azure DevOps services and descriptions

Service Name	**Description**
Azure Artifacts	A service that enables the sharing of build packages and deployment of projects, supporting various package formats such as Maven, npm, NuGet, and more.
Azure Boards	A collection of tools designed for planning and managing work on a project, utilizing Kanban and Scrum methodologies to facilitate task tracking, progress monitoring, and team collaboration.
Azure Pipelines	A service that facilitates the automated creation and testing of projects, allowing for streamlined Continuous Integration (CI) and Continuous Deployment (CD) workflows to ensure efficient software delivery pipelines.
Azure Repos	A version control system that offers support for both Git and Team Foundation Version Control (TFVC) enables teams to manage source code effectively, collaborate on development, and track changes to codebases throughout the project lifecycle.
Azure Test Plans	A suite of application testing tools encompassing various testing methodologies, including continuous testing, to ensure the quality and reliability of software applications through comprehensive testing practices.

2.6.4 Azure Pipeline

From the study's objectives perspective, the primary focus lies in analyzing the Azure Pipelines service. As highlighted earlier, Azure Pipelines offers automatic project creation and testing, catering to developers' needs by supporting solutions in various programming languages, such as Python, Java, JavaScript, PHP, Ruby, C#, C++, and Go (Eloranta, 2018). Additionally, it enables code deployment to diverse targets, including virtual machines, environments, containers, local and cloud platforms, and PaaS offerings. The service seamlessly integrates with GitHub and facilitates continuous testing, allowing projects to be

built on multiple operating systems such as Windows, Linux, and Mac. Furthermore, it enables the creation and deployment of containers in Docker and Kubernetes environments.

At the core of Azure Pipelines lie the principles of CI and CD. According to Bondarenko (2020), CI is a software development practice that involves constantly merging working copies into a single branch, facilitating rapid identification and resolution of code integration issues while ensuring developers have access to the current stable version of the project alongside working branches. Conversely, CD is a practice that automates a project's building, testing, and deployment whenever changes occur. This results in continuously deploying artifacts generated within the CI process, ensuring software is continually updated and promptly addresses any identified deficiencies.

A fundamental concept within Azure Pipelines is the pipeline, which, according to Saarenpää (2020), represents a sequential process of continuous integration and deployment within the framework of CI and CD. The pipeline comprises stages, each delineating a specific process phase and may encompass one or more tasks. The smallest unit within the pipeline is the step, which can be a script or a task initiating a particular build, test, or deployment action for the project. Regarding CD implementation, the trigger assumes paramount importance in the pipeline configuration. Here, a trigger serves as a condition under which the pipeline executes, allowing projects to initiate new builds or deployments based on predefined conditions.

2.6.5 Deploying Azure Resources with Bicep

Azure Bicep is a DSL (domain-specific language) built specifically for deploying Azure resources (Barać & Scott-Raynsford, 2023). As mentioned above, creating bicep was to simplify the creation of Azure Resource Manager templates. Flowers (2023) affirms that bicep can be considered an alternative to creating templates using JSON and terraform in the declaration area of Azure resources. The main features of the bicep in literature are as follows.

Table 4 presents main features of Bicep include its declarative nature, Azure-centric focus, translation to Azure Resource Manager templates, simplified resource declaration with concise syntax, and integration with Azure Pipelines for automated deployments.

Table 4: Main features of bicep

Feature	**Description**
Declarative Nature	Bicep is declarative, allowing flexible declaration of Azure resources without requiring strict ordering of elements.
Azure-Centric Focus	Bicep is tailored for Azure, focusing solely on solving deployment challenges within the Microsoft Azure ecosystem.
Translation to Azure Resource Manager Templates	Bicep files are translated into Azure Resource Manager JSON templates for deployment, simplifying the creation process without replacing Azure Resource Manager templates.
Simplified Resource Declaration	Bicep offers a concise and readable syntax for declaring Azure resources, making infrastructure as code management easier.
Integration with Azure Pipelines	Bicep seamlessly integrates with Azure Pipelines, enabling the automated deployment of Azure resources defined in Bicep files and enhancing DevOps processes.

Bicep offers simple syntax, strong security, and extensive code reuse (L’Esteve, 2023). Unlike JSON, Bicep's syntax is more readable, concise, and simpler than HashiCorp's, making deployment easier for developers. The comparison between Table 2 Bicep syntax and JSON and HashiCorp (HCL) follows.

Table 5 presents Syntax comparisons for Bicep, JSON, and HashiCorp Language (HCL) cover parameter creation, resource declaration, loops, conditional operators, and variable declaration, showcasing their respective syntax approaches.

Table 5:Bicep, JSOn, and HashiCorp (HCL) syntax

Creating a parameter (on the example of resource location)	
bicep	param location string = resourceGroup().location

JSON	"location": {
	"type": "string",
	"defaultValue": "[resourceGroup().location]"
	}

HCL	location = azurerm_resource_group.location
Resource declaration	

bicep	resource symbolicname 'Microsoft.Storage/storageAccounts@2021-09-01' = {
	...
	}

JSON	"resources": [
	{
	"type": "Microsoft.Storage/storageAccounts",
	"apiVersion": "2021-09-01",
	...
	}
	]

HCL	resource "azurerm_storage_account" "resourse_name" {
	...
	}

Loops (iterate over elements in an array)	
Bicep	[for storageName in storageAccountNamesArray: { ... }]
JSON	"copy": { "name": "storagecopy", "count": "[length(parameters(' storageAccountNamesArray'))]" }, ...
HCL	for_each = toset(var.storageAccountNamesArray)

	...
Conditional (ternary) operators	
Bicep	isTrue ? 'valueIfTrue' : 'valueIfFalse'
JSON	[if(parameters('isTrue'), 'valueIfTrue,' 'valueIfFalse')]
HCL	isTrue ? 'valueIfTrue' : 'valueIfFalse'
Variable declaration	
Bicep	var test = 'example value'
JSON	"variables": { "test": "example value" },

HCL	locals {
	  test = 'example value'
	}

3 Methodology

The methodology chapter outlines a mixed-methods approach to investigate Infrastructure as Code in Azure, including research design, sampling, data collection, analysis methods, ethical considerations, and study limitations.

3.1 Research Design

A combination of scientific methods, field experiment approach, and primary research was adopted in this study to investigate the feasibility of Infrastructure as Code in Azure infrastructure provisioning and determine its impact on the ease of use of the Azure platform and its acceptance. Specifically, the first part of the project was coding Infrastructure as Code, after which Azure clients and consultants were recruited for a field experiment to score their perceived ease of use of the Azure platform with and without the developed Infrastructure as Code. The objective was to test whether a significant difference existed in ease of use when deploying infrastructure on Azure using Infrastructure as Code and the existing solutions or manual configuration. Besides, interviews were conducted with IT professionals to understand their concerns about using Infrastructure as Code in infrastructure provisioning and management.

Both qualitative and quantitative study approaches were adopted. The quantitative aspect of the study involved collecting data from the case involving using Infrastructure as Code syntax coded in this study and manual approach when provisioning infrastructure on Azure. On the other hand, the qualitative part of the study was aimed at giving the quantitative research findings context. A mixed approach in the study was necessary, accompanied by a field experiment approach, which was vital in providing comprehensive results and understanding of the subject matter. The quantitative part of the study focused on the analysis of variance to determine the impact of Infrastructure as Code as a factor in revolutionizing the modern digital landscape.

The implementation of the research objectives necessitated the use of a whole group of scientific methods. The use of general logical methods (analysis, synthesis, abstraction) made it possible to study the theoretical aspects of the implementation of CD / CI in MS Azure, consider the experience of previous researchers of the problem, and summarize the information contained in the official MS Azure documentation. The result was implementing a comprehensive analysis of CD / CI features in working with MS Azure services. The

comparative method was of great importance in the context of the implementation of the research objectives. The comparative analysis allowed testing the hypothesis that bicep has a simpler syntax and is more convenient for the developer and user than JSON and, at the same time, has many advantages compared to HashiCorp (in particular, it supports new versions of Azure resources).

The field experiment approach allowed for testing and validating the hypothesis that programmatic deployment of Azure services using CI/CD technologies has many advantages over the traditional use of cloud management through the user interface. As part of the experiment, creating an Azure Resource Manager template using the bicep declarative language and using them as Infrastructure as Code allowed deployed many Azure services, including virtual machines, databases, and blob storages.

3.2 Sampling, Data, and Data Collection

When completing tasks in this subsection, the material management plan presented in Appendix 1 was followed. The target group consisted of 20 Azure clients and consultants, selected through purposive sampling to ensure participants were well-acquainted with the Azure portal. These participants were recruited from organizations that regularly use Azure for their cloud infrastructure needs. They were selected based on their roles, which typically involve tasks such as deploying, managing, and maintaining cloud services on Azure. The 20 Azure clients and consultants were recruited for field experiment, data collection, and interview purposes based on questions presented in Appendix 2.

Each participant was tasked with using both the coded Infrastructure as Code and the manual process for provisioning infrastructure on Azure. They recorded data on the time taken, perceived ease of use, error rates, consistency, and task success rates. These tasks involved setting up virtual machines, databases, and blob storages, allowing for a comprehensive comparison of the two approaches. This detailed selection and task assignment aimed to ensure that the data collected was relevant and reflective of real-world scenarios, providing robust insights into the feasibility and impact of Infrastructure as Code in Azure provisioning.

They were also asked to use the coded Infrastructure as Code and manual process, record data on time used to provision infrastructure on Azure, and their perceived ease of use when using Infrastructure as Code and manual approach. Data on error, consistency, and task success rates was collected. Once data was collected, open-ended interviews and surveys

were conducted to ascertain the challenges the participants might have encountered when using Infrastructure as Code in infrastructure provisioning on the Azure platform.

3.3 Analysis

T-tests were adopted in the study. The objective was to ascertain how the independent variable (whether infrastructure as a code was used) impacted dependent variables. Analysis of variance was vital in this study to determine if Infrastructure, as Code, solved some of the challenges exhibited when adopting a manual approach to application deployment on Azure. The dependent variables were perceived ease of use, task success rate, error rate, and consistency rate. On the other hand, the independent variable deployment approach takes two levels (manual infrastructure provisioning and Infrastructure as Code deployment).

3.4 Ethical Considerations

While this study involves coding and testing Infrastructure as Code, it also involves study participants, making it necessary to consider ethical research practices to protect participants' rights. First, consent was obtained from all participants, and they were allowed to leave the study at any time of their choice. Besides, their privacy was safeguarded by ensuring anonymity. Their personal identification information was not collected, and their data was used solely for this study. The study's design was aimed at validating the reliability and validity of the study while minimizing conflict of interest. Cronbach's alpha was used to investigate the study's validity, and any potential limitations were reported to inform readers of its shortcomings.

3.5 Study Limitations

Despite the study providing a solution to challenges associated with using manual approaches to infrastructure commissioning, it exhibits limitations that readers should note. A small sample size was used that might not reflect the population. This limits the generalizability of the study, whereas purposive sampling introduces inherent selection biases that limit the study's reliability. Besides, selecting people who were well acquainted with the Azure portal limits the study from investigating those less familiar and experienced with the cloud computing platforms.

4 Project Implementation

The project implementation section details the systematic process of deploying Azure infrastructure using Bicep templates. It covers template design, resource declarations, deployment scripts, and pipeline implementation, emphasizing Bicep's advantages in simplifying infrastructure management and enhancing deployment efficiency.

4.1 Project Design

The design and implementation of the project using Bicep templates involved a systematic and iterative process. In this study, Bicep was adopted for the Azure infrastructure deployment solution. First, the requirements and objectives of the project were defined. This included identifying the desired Azure resources, their configurations, and any specific dependencies or constraints. The project scope was determined by considering scalability, security, and cost optimization factors. Next, the Bicep template was designed to describe the desired state of the Azure infrastructure. This involved creating the necessary resource definitions, specifying properties, and establishing relationships between resources. The Bicep language provided a concise and declarative syntax, allowing for easier comprehension and maintenance of the template code.

4.2 Bicep Templates

Figure 1 shows the Azure root directory, providing insights into the project’s tree for Azure Resource Manager templated deployment based on Infrastructure as a Code syntax. In the tree, Azure Resource Manager was used to define resources. In contrast, the JSON syntax specified the configurations in this project that were required to be deployed on the Azure platform. The files in the project's tree were azure-pipelines.yml, used to define continuous integration or continuous delivery; bicepconfig.json, used to specify configurations for Bicep; main.bicep, which was used for resources used for Azure deployment and main.parameters.dev.json, which harbored parameters used to customize deployments based on specific user requirements and needs. Adopting the tree file structure presented in Figure 1 allowed the creation of more modular deployment.

Figure 1: Program code 1 highlighting project tree

```
deploy
azure-pipelines.yml
bicepconfig.json
main.bicep
main.parameters.dev.json
```

As part of solving this task, a main.bicep file was created, and a description of the resource to be deployed to the Azure storage account was provided. Bicep is a declarative language where the order of the elements does not affect the deployment process. At the same time, in the bicep file, it is customary to define the application parameters first. Figure 2 illustrates a few options when deploying the application, including location, keys, and an array of container names used to deploy multiple blob storage using a bicep loop. It is worth noting that when declaring the parameters, decorators are used, which simplify the implementation of restrictions and metadata compared to Azure Resource Manager templates written using JSON. The description decorator contains a description of the parameters that users can see through the portal.

Figure 2: Program code 2 highlighting project resource description

```
@description('Specifies the location in which the Azure Storage resources should be deployed.')
param location string = resourceGroup().location
```

Figure 3 shows the code used to set decorators minLength and maxLength, which set the minimum and maximum integer values of the string or array bicep length. The code also presents the code used to create a globally unique identifier and the parameter values provided.

Figure 3: Program code 3 highlighting setting décor parameters and global unique identifier

```
@minLength(5)
@maxLength(30)
param storageAccountName string = 'Client${uniqueString(resourceGroup().id)}'
@description('Specifies the name of the blob container!!!!!!')
```

The next step involved securing decorator parameters that contain information about application keys to protect the proposed system from security vulnerabilities. To accomplish this, the decorated settings were not stored or logged in the deployment log; instead, they were secured, as shown in Figure 4.

Figure 4: Program code 4 highlighting securing parameters

```
@secure()
param keyVaultName string
@secure()
param keyVaultAccessObjectID string
@secure()
param applicationId string
@description('The name and tier of the App Service plan SKU.')
param keyVaultPlanSku object
```

Further, variables were created to get and store the value of the sasToken. First, the accountSasProperties helper variable was created and initialized, as presented in Figure 5. This variable contains the dependency object needed to get the sasToken. Next, the value of sasToken was retrieved using the bicep listAccountSas() resource function, and its value was stored in a variable.

Figure 5: Program code 5 highlighting getting, storing, and retrieving the value of the sasToken

```
var accountSasProperties = {
    signedServices: 'b'
    signedPermission: 'r'
    signedExpiry: '2023-08-20T11:00:00Z'
    signedResourceTypes: 's'
}
var sasToken = storageAccount.listAccountSas(storageAccount.apiVersion, accountSasProperties).accountSasToken
```

Moving forward, StorageAccount deployment was declared. An Azure storage account contains all Azure Storage data objects, including blobs, file shares, queues, tables, and disks. A storage account is a container that keeps Azure storage services together. Integrating data services into a storage account allows users to manage them as a group. Figure 6 shows how the name, local, kind, and SKU were declared. The SKU name is Standard_LRS (Standard Locally Redundant Storage). The type selected as a kind is StorageV2 (a general-purpose account that supports the latest Azure storage features and includes all the features of v1 general-purpose accounts and blob storage). StorageAccount dependencies were also specified (accessTier, allowCrossTenantReplication, and others).

Figure 6: Program code 6 highlighting the name, local, kind, and SKU declaration, and Storage Account dependencies specification

```
resource storageAccount 'Microsoft.Storage/storageAccounts@2021-04-01' = {
  name: storageAccountName
  location: location
  sku: {
    name: 'Standard_LRS'
  }
  kind: 'StorageV2'
  properties: {
    accessTier: 'Hot'
    //defaultToOAuthAuthentication: false
    allowCrossTenantReplication: false
    minimumTlsVersion: 'TLS1_2'
    allowBlobPublicAccess: true
    allowSharedKeyAccess: true
    isHnsEnabled: true
  }
}
```

Once the StorageAccount was deployed, the next step involved declaring several Azure BlobStorages containers, as presented in Figure 7. These containers are designed to store large amounts of unstructured data, primarily text or binary files. The declaration was done using a for-in bicep loop, as highlighted in Figure 7.

Figure 7: Program code 7 highlighting BlobStorages declaration using bicep loop

```
resource blobService 'Microsoft.Storage/storageAccounts/blobServices/containers@2021-04-01' = [for name in containersNames:  {
  name: format('{0}/default/{1}',storageAccountName, name)
  dependsOn: [
    storageAccount
  ]
}]
```

Moving forward, the script involved creating a blobStorage for each name contained in the containerNames array declared as a parameter within the loop presented in Figure 7. Considering that using the containers depends on the property, the so-called direct dependence of blob storage on StorageAccount was declared, as presented in Figure 8.

Figure 8: Program code 8 highlighting creating a blobStorage for each containerNames

```
param containersNames array = [
  'edi'
  'ediarch'
  'factoring'
  'rfpayment'
  'rfpaymentarch'
  'statement'
  'statementarch'
  'sepa'
  'finvoice'
]
```

Once the storage containers were deployed, the following script covered the services required to secure the application. Azure Key Vault is a service for securely storing and accessing secrets (API keys, passwords, certificates, cryptographic keys). Access policies were declared as a service, as evident in Figure 9. As part of policy configuration, permissions were given to get keys and get and set secrets for the application with the specified ID. Besides, soft delete was disabled, and other useful features were enabled, as presented in Figure 9.

Figure 9: Program code 9 highlighting services deployment to secure cloud application

```
resource keyVault 'Microsoft.KeyVault/vaults@2021-10-01' = {
  name: keyVaultName
  location: location
  tags: {
    tagName1: keyVaultName
  }
  properties: {
    accessPolicies: [
      {
        applicationId: applicationId
        objectId: keyVaultAccessObjectID
        permissions: {
          keys: [
            'get'
          ]
          secrets: [
            'list'
            'get'
            'set'
          ]
        }
        tenantId: subscription().tenantId
      }
    ]
    enabledForDeployment: true
    enabledForDiskEncryption: true
    enabledForTemplateDeployment: true
    enableSoftDelete: false
    sku: keyVaultPlanSku

    tenantId: subscription().tenantId
  }
}
```

Next, the parent property was used to indicate that a resource was a child of keyVault to create a secret to store the sasToken, as evident in Figure 10.

Figure 10: Project code 10 highlighting the creation of a secret to store the sasToken

```
resource keyVaultSecrets 'Microsoft.KeyVault/vaults/secrets@2021-10-01' = {
  name: keyVaultName
  parent: keyVault
  properties: {
    value: sasToken
  }
}
```

Having completed scripting the Bicep template, the next step was to deploy Azure Key Vault using data stored in the deploymentParameters.json file, as presented in Figure 11.

Figure 11: Program code 11 highlighting the parameter JSON scrip used in deploying the Azure Key Vault

```
1  {
2      "$schema": "https://schema.management.azure.com/schemas/2019-04-01/deploymentParameters.json#",
3      "contentVersion": "1.0.0.0",
4      "parameters": {
5        "keyVaultPlanSku": {
6          "value": {
7              "family": "A",
8              "name": "standard"
9          }
10       },
11       "keyVaultName": {
12         "value": "testikeyvault1234567"
13       },
14       "keyVaultAccessObjectID": {
15         "value": "[redacted]"
16       },
17       "applicationId": {
18         "value": "[redacted]"
19       }
```

4.3 Pipeline

Once Azure Key Vault deployment was completed, Next, resources were deployed using Pipeline YAML. The Azure Resource Manager template file is the main one. The bicep file was compiled using the JSON language. Next, the bicep file was tested and validated by executing the YAML file in the PowerShell script.s. This marked the final stage of deploying MS Azure services. YAML is a convenient data serialization language often used to create configuration files with other programming languages. YAML is a strict overlay of JSON for serialization designed for human interaction. However, because of its strict superset, it can do everything JSON can and more. One of the main differences is that newlines and indentation mean something in YAML, unlike JSON, which uses square brackets and curly braces.

For the pipeline implementation, a trigger was scripted to deploy every time changes are pushed to Git, as presented in Figure 12. Following this approach for the Infrastructure as a Code implementation was important as it bypasses the need to go to Azure DevOps, which requires manual intervention to start, thereby saving time. Windows were selected for this deployment field experiment, after which the pipeline stage was scripted following these stages. Completing Bicep linter to Bicep files for syntax errors and best practice violations. The linter helps enforce coding standards by guiding development, as presented in Figure 12.

Figure 12: Program code 12 highlighting trigger and bicep linter implementation script

```yaml
trigger:
  - '*'

pool:
  vmImage: 'windows-latest'

stages:

- stage: Lint
  jobs:
  - job: LintCode
    displayName: Lint code
    steps:
      - script: |
          az bicep build --file deploy/main.bicep
        name: LintBicepCode
        displayName: Run Bicep linter
```

The next step was taken to handle the linter warning. Considering Azure Pipelines does not treat linters warnings as mistakes, any mistakes it presents do not stop the pipeline. As a result, the bicepconfig.json file presented in Figure 13 was used to customize and reconfigure it.

Figure 13: Program code 13 highlighting bicepconfig.json reconfiguration

```
deploy > bicepconfig.json > ...
{
  "analyzers": {
    "core": {
      "enabled": true,
      "verbose": true,
      "rules": {
        "no-hardcoded-env-urls": {
          "level": "error"
        },
        "no-unused-params": {
          "level": "error"
        },
        "no-unused-vars": {
          "level": "error"
        },
        "prefer-interpolation": {
          "level": "error"
        },
        "secure-parameter-default": {
          "level": "error"
        },
        "simplify-interpolation": {
          "level": "error"
        }
      }
    }
  }
}
```

Once the bicep linter was reconfigured, the next step was validation, completed in the script presented in Figure 14. The bicep code was validated in the deployment pipeline to prevent syntax errors, as shown in Figure 14.

Figure 14: Program code 14 highlighting bicep code validation

```
19  - stage: Validate
20    jobs:
21    - job: ValidateBicepCode
22      displayName: Validate Bicep code
23      steps:
24        - task: AzureCLI@2
25          name: RunPreflightValidation
26          displayName: Run preflight validation
27          inputs:
28            azureSubscription: $(ServiceConnectionName)
29            scriptType: 'bash'
30            scriptLocation: 'inlineScript'
31            inlineScript: |
32              az deployment group validate \
33                --resource-group $(ResourceGroupName) \
34                --template-file deploy/main.bicep \
35                --parameters deploy/main.parameters.dev.json
```

Once the code was validated, the next step encompassed deploying the template directly from a repository, as presented in the script in Figure 15.

Figure 15: Program code 15 highlighting the deployment script

```
38  - stage: Deploy
39    jobs:
40    - job: Deploy
41      steps:
42      - task: AzureCLI@2
43        inputs:
44            azureSubscription: $(ServiceConnectionName)
45            scriptType: 'bash'
46            scriptLocation: 'inlineScript'
47            inlineScript: |
48              az deployment group create \
49                --name $(Build.BuildNumber) \
50                --resource-group $(ResourceGroupName) \
51                --template-file deploy/main.bicep \
52                --parameters deploy/main.parameters.dev.json
```

Program code 16, presented in Figure 16, marks the end of the pipeline implementation by publishing the built artifact to the Azure pipeline.

Figure 16: Program code 16 highlighting script that publishes built artifact to Azure pipeline

```
- stage: Build
  jobs:
  - job: Build
    steps:
    - task: CopyFiles@2
      displayName: 'Include templates in the artifact'
      inputs:
        sourceFolder: 'deploy'
        targetFolder: '$(Build.ArtifactStagingDirectory)'
    - task: PublishBuildArtifacts@1
      displayName: 'Publish Artifact: drop'
      inputs:
          PathtoPublish: '$(Build.ArtifactStagingDirectory)'
          ArtifactName: 'drop'
          publishLocation: 'Container'
```

The process presented in program code 16 is not instant but takes a few minutes, which in this field experiment case was approximately four minutes and twelve seconds. As evident in the output in Azure DevOps, the developed Infrastructure as a Code program completed all the scripted tasks without any problems or errors, as evident in the deployment logs.

4.4 Azure DevOps

Figure 17 presents the steps followed in the Infrastructure as a Code, encompassing lint, validation, deployment, and building, whereas Figure 18 presents the list of jobs.

Figure 17: The program Infrastructure as a Code stages

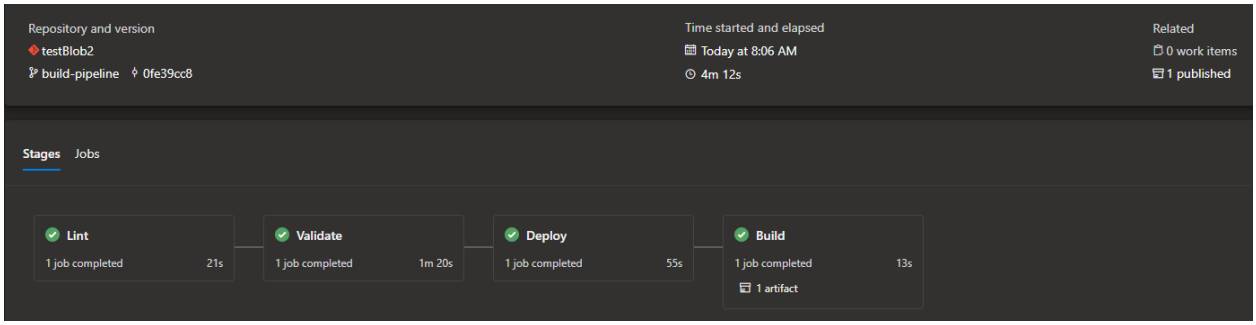
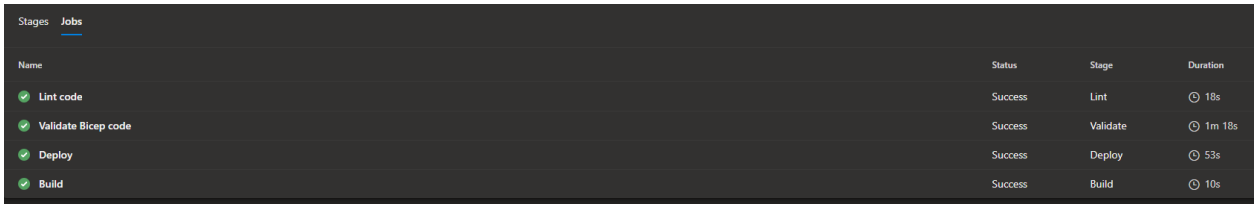


Figure 18: List of jobs



After implementing the previous steps, it was evident, as shown in Figure 17 and Figure 18, that the artifact was available. In the jobs folder presented in Figure 17, bicep code, config, parameters, and YAML were evident. As a result, the Infrastructure as a Code used in this study will be used in the CI/CD pipelines in the future using the artifacts presented in Figure 19.

Figure 19: Artifacts

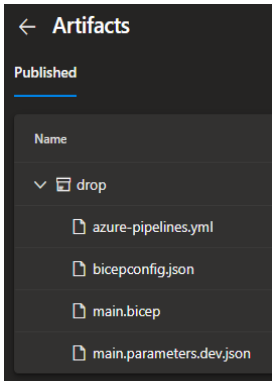
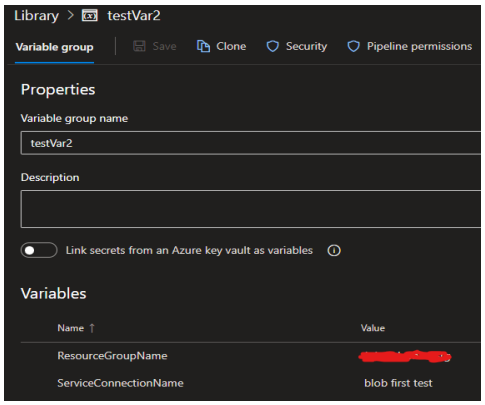


Figure 20 shows an example of a pipeline library used to store the values and secrets used in the pipeline. The library can also be used in multiple pipelines and is protected as a resource.

Figure 20: Pipeline library used to store the values and secrets



Release pipelines were used to deliver software faster and with less risk, as presented in the script in Figure 21. Multiple steps were also used to automate production or set up semi-automated processes with approvals and on-demand deployments.

Figure 21: Release Pipeline

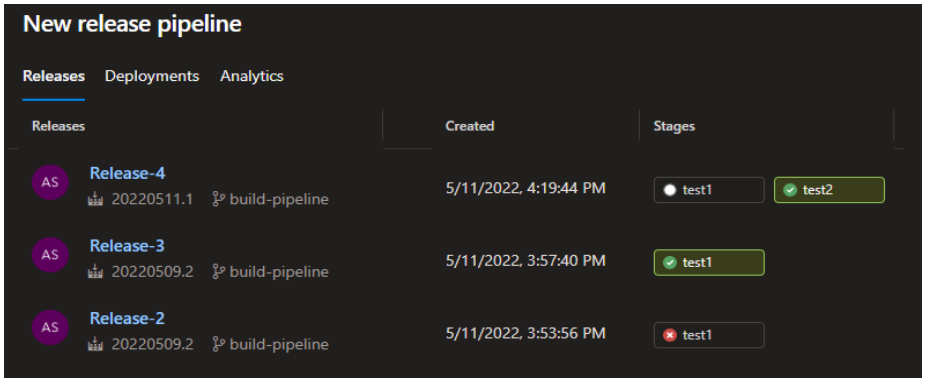
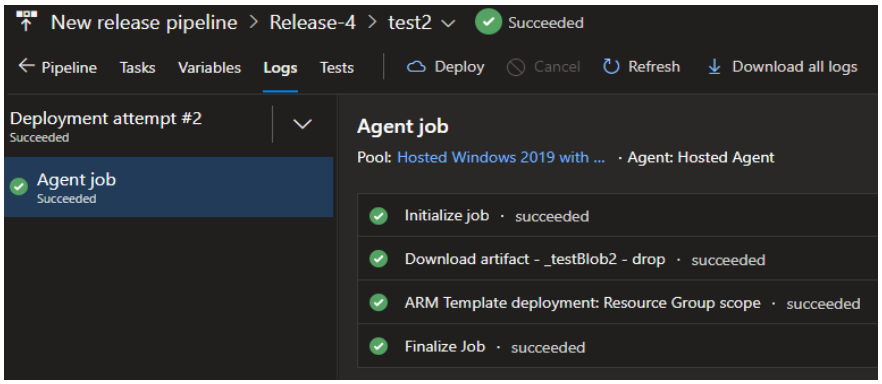


Figure 22 shows the output generated once the pipeline script runs without errors on the proposed Infrastructure as code.

Figure 22: Output generated after successfully running the pipeline scripts



Overall, once the pipeline process was complete, a working infrastructure with working resources was deployed in the Azure cloud. All containers and the key vault were deployed. Figure 23 shows the deployment details.

Figure 23: Successful deployment details

Deployment details

Resource	Type	Status
testikeyvault1234567/testikeyvault1234567	Microsoft.KeyVault/vaults/secrets	OK
octoberh2dj52zjrjwgo	Microsoft.Storage/storageAccounts	OK
octoberh2dj52zjrjwgo/default/statementarch	Microsoft.Storage/storageAccounts/blobServices/containers	Created
octoberh2dj52zjrjwgo/default/factoring	Microsoft.Storage/storageAccounts/blobServices/containers	Created
octoberh2dj52zjrjwgo/default/rfpayment	Microsoft.Storage/storageAccounts/blobServices/containers	Created
octoberh2dj52zjrjwgo/default/finvoice	Microsoft.Storage/storageAccounts/blobServices/containers	Created
octoberh2dj52zjrjwgo/default/edi	Microsoft.Storage/storageAccounts/blobServices/containers	Created
octoberh2dj52zjrjwgo/default/statement	Microsoft.Storage/storageAccounts/blobServices/containers	Created
octoberh2dj52zjrjwgo/default/sepa	Microsoft.Storage/storageAccounts/blobServices/containers	Created
octoberh2dj52zjrjwgo/default/rfpaymentarch	Microsoft.Storage/storageAccounts/blobServices/containers	Created
octoberh2dj52zjrjwgo/default/ediarch	Microsoft.Storage/storageAccounts/blobServices/containers	Created
octoberh2dj52zjrjwgo	Microsoft.Storage/storageAccounts	OK
testikeyvault1234567	Microsoft.KeyVault/vaults	OK

In the final step, a Logic app workflow was deployed to track BlobStorage changes, including file creation, content retrieval, and content deletion. The Logic app workflow necessarily includes a trigger that specifies a condition for performing actions within the workflow and actions that define a specific sequence of actions. Three parameters that store the name of the deployed Logic app service, location, and test Url were created as part of the deployment, as shown in Figure 24.

Figure 24: Program code 17 highlighting the three created parameters

```
@description('The name of the logic app to create.')
param logicAppName string = 'Main${uniqueString(resourceGroup().id)}'

@description('A test URI')
param testUri string = 'https://status.azure.com/en-us/status/'

@description('Location for all resources.')
param location string = resourceGroup().location
```

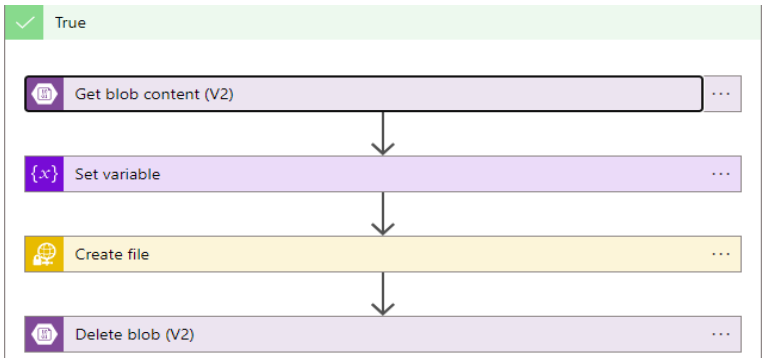
From then on, the three parameters created in Figure 24 were used when deploying the workflow presented in Figure 18.

Figure 25: Program code 18 highlighting workflow deployment

```
resource stg 'Microsoft.Logic/workflows@2019-05-01' = {
  name: logicAppName
  location: location
  identity: {
    type: 'SystemAssigned'
  }
  tags: {
    displayName: logicAppName
}
```

Figure 26 shows the logic deployed in Azure. Navigating into each deployed window was necessary to get the output and see more detailed settings.

Figure 26: Logic app

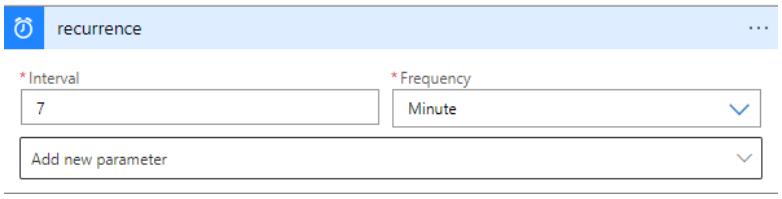


Despite deploying the logical app, further steps were taken to ensure the proposed Infrastructure as Code was optimized to outperform current solutions. Following this, variables that store the trigger type and settings were created, as shown in Figure 27. The specified settings use a recurrence trigger as a trigger type, which will run the workflow with a frequency of 7 minutes, as shown in Figure 28.

Figure 27: Program, code 19 highlighting trigger type and settings variables

```
var filenameOut = ''
var frequency = 'Minute'
var interval = 7
var type = 'recurrence'
var actionType = 'ApiConnection'
var workflowSchema = 'https://schema.management.azure.com/providers/Microsoft.Logic/schemas/2016-06-01/workflowdefinition.json#'
```

Figure 28: Recurrence frequency



Besides, the action type was specified. The ApiConnection type used assumes that an HTTP request is made to a Microsoft-managed API and requires a reference to a valid API connection and information about its parameters, as presented in Figure 29.

Figure 29: Program code 29 showing action type definition

```
definition: {
    '$schema': workflowSchema
    contentVersion: '1.0.0.0'
    parameters: {
        filenameOut: {
            type: 'string'
            defaultValue: filenameOut
        }
        testUri: {
        type: 'string'
        defaultValue: testUri
        }
        '$connections': {
        defaultValue: {}
        type: 'Object'
        }
    }
        triggers: {
            recurrence: {
            type: type
                recurrence: {
                    frequency: frequency
                    interval: interval
                }
```

The API connection links were declared in the definition Workflow block, and their values were set in the properties section, as shown in Figure 30.

Figure 30: Program code 21 highlighting API connections declaration

```
properties: {
    parameters: {
        '$connections': {
            value: {
                azureblob: {
                    connectionId: '/subscriptions/${subscription().subscriptionId}/resourceGroups/${location}/providers/Microsoft.Web/connections/azureblob-1'
                    connectionName: 'azureblob-1'
                    connectionProperties: {
                        authentication: {
                            type: 'ManagedServiceIdentity'
                        }
                    }
                    id: '/subscriptions/${subscription().subscriptionId}/providers/Microsoft.Web/locations/westeurope/managedApis/azureblob'
                }
                sftpwithssh: {
                    connectionId: '/subscriptions/${subscription().subscriptionId}/resourceGroups/${location}/providers/Microsoft.Web/connections/sftpwithssh-10'
                    connectionName: 'sftpwithssh-10'
                    id: '/subscriptions/${subscription().subscriptionId}/providers/Microsoft.Web/locations/westeurope/managedApis/sftpwithssh'
```

Figure 31 shows the API connection established in the field experiment associated with the logical app.

Figure 31: API connections associated with the logical app output

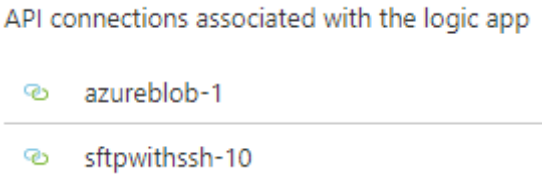


Figure 32 shows the script for the actions block defining specific workflow implementation steps. Action types such as foreach and if were used as presented in program code 22. Foreach involves a looping action that iterates through an array and executes certain instructions on each element. As part of the logic, the actions "Create_file," "Delete_blob_(V2)", "Get_blob_content_(V2)," and "Set_variable" were executed for each Blob Storage directory (an array of directories was obtained as a result of executing an action named Lists_blobs_(V2)). Executing this function resulted in the output presented in Figure 33.

Figure 32: Program code 22 highlighting actions blocks definitions

```
actions: {
    For_each: {
        foreach: '@body(\'Lists_blobs_(V2)\')?[\'value\']'
        runAfter: {
            'Lists_blobs_(V2)': [
                'Succeeded'
            ]
        }
```

Figure 33: List of blobs

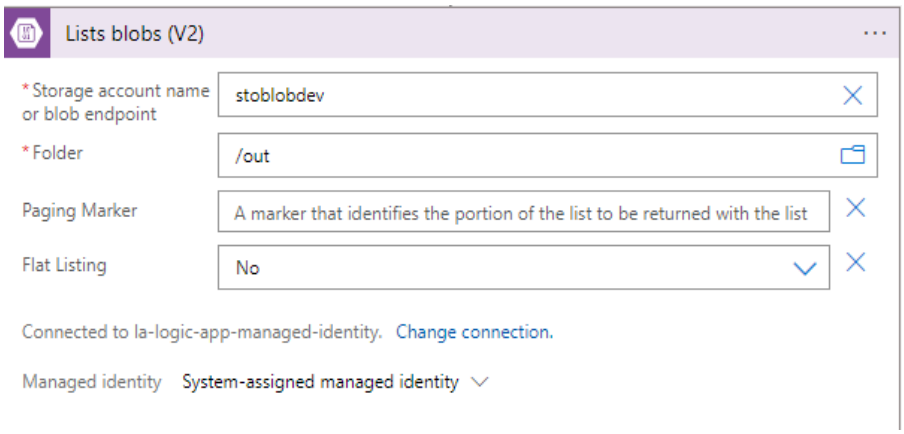


Figure 34 shows the control structure that compares the data in the workflow with its values or fields that have been expanded in our code.

Figure 34: Program code 23 showing the control structure that compares the data in the workflow

```
type: 'Foreach'
actions: {
    Condition: {
        actions: {
            Create_file: {
                runAfter: {
                    Set_variable: [
                        'Succeeded'
```

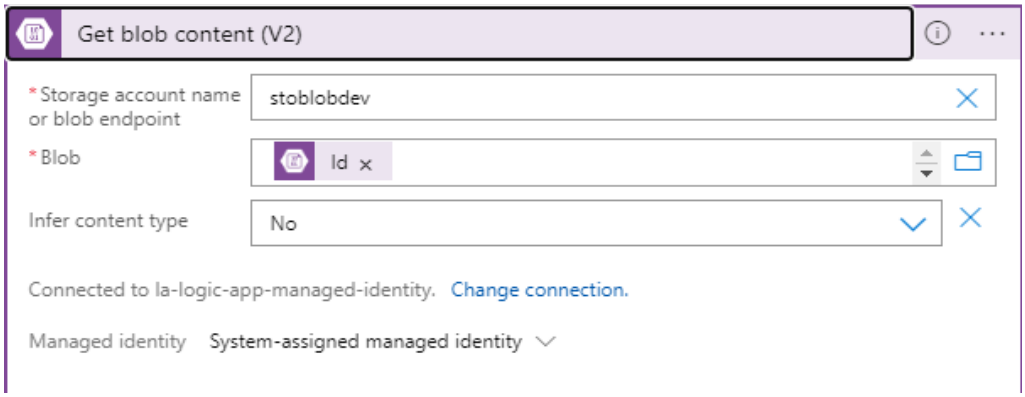
Proceeding, the condition (in this case, whether the BlobStorage element is a directory) is specified in the expression section presented in Figure 35.

Figure 35: Program code 24 highlighting condition expression in the specified expression

```
expression: {
    and: [
        {
            equals: [
                '@items(\'For_each\')?[\'IsFolder\']'
                false
            ]
        }
    ]
}
runAfter: {}
type: 'If'
```

Based on the insight, the actions presented in this subsection ("Create_file," "Delete_blob_(V2)," and "Get_blob_content_(V2)") only work on array elements that are directories. Figure 36 shows the interface in Dev Ops used to get the blob. Without conditions, the actions initialize variables, set the name of the output file "Lists_blobs_(V2)", and get the content of the BlobStorage as an array, are executed. The inputs block in the action contains a link to the ApiConnection connection and information about its parameters previously defined in the $connections parameter as the reference value for connecting to the ApiConnection.

Figure 36: Getting blob



Besides, ApiConnection parameters can include method, host, and headers as shown in Figure 37.

Figure 37: Program code 25 Action type ApiConnection parameters

```
type: actionType
inputs: {
    host: {
        connection: {
            name: '@parameters(\'$connections\')[\'azureblob\'][\'connectionId\']'
        }
    }
    method: 'get'
    path: '/v2/datasets/@{encodeURIComponent(encodeURIComponent(\'stoblobdev\'))}/foldersV2/@{encodeURIComponent(encodeURIComponent(\'/out\'))}'
    queries: {
        nextPageMarker: ''
        useFlatListing: false
```

The program code presented in code 26 shows the script configuring receiving fragmented messages.

Figure 38: Program code 26 highlighting fragmented messages configuration

```
runtimeConfiguration: {
    contentTransfer: {
        transferMode: 'Chunked'
    }
}
```

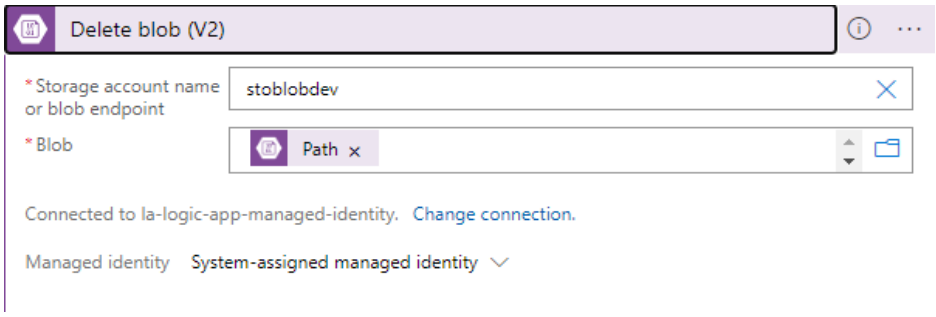
The runAfter property lists the actions that must run the current action on successful completion. In this case, the "Delete_blob" action was implemented and executed after the file was successfully created, as shown in program 27.

Figure 39: Program code 27 used to delete the blob

```
'Delete_blob_(V2)': {
    runAfter: {
        Create_file: [
            'Succeeded'
        ]
    }
```

Figure 40 below shows the delete blog logic using the Azure portal interface on the Azure platform. The interface involves defining the account name or endpoint and blob path.

Figure 40: Delete blob in Azure logic



Similarly, the "Set_Variable" action is executed only after the content of the Blob Storage has been received, as presented in Figure 41. Figure 42 shows the Azure interface used for the “Set_Variable” action, whereas Figure 43 shows the code used to initialize the action variable without predefined conditions.

Figure 41: Program code 28 highlighting the script used to implement the “Set Variable” action

```
Set_variable: {
    runAfter: {
        'Get_blob_content_(V2)': [
            'Succeeded'
        ]
    }
    type: 'SetVariable'
    inputs: {
        name: 'filenameOut'
        value: '@items(\'For_each\')?[\'Name\']'
    }
```

Figure 42: Interface for setting variable

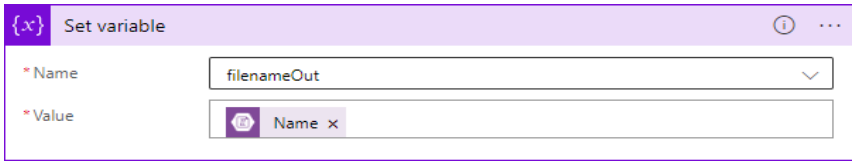
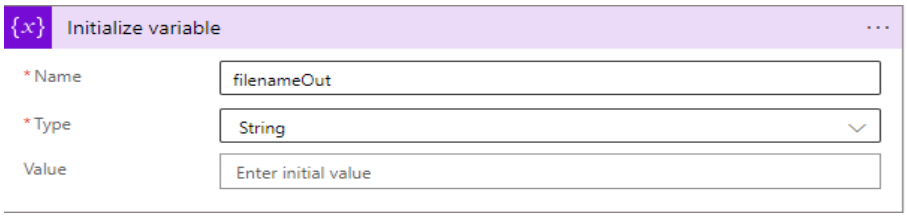


Figure 43: Program code 29 highlighting for initializing variable

```
Initialize_variable: {
                runAfter: {}
                type: 'InitializeVariable'
                inputs: {
                    variables: [
                        {
                        name: 'filenameOut'
                            type: 'string'
                        }
                        ]
                        }
```

Finally, the following interface from Azure was leveraged to initialize the variable, as presented in Figure 44.

Figure 44: Initializing variable



Using the 'for each,' 'if,' and 'run after' properties allowed the following logic to be built for working with BlobStorage content. The action "Initialize_variable" is executed first. After its completion, the action "Lists_blobs_(V2)" gets the content of the BlobStorage as an array. This array is looped over and provided that the element is a directory, the actions "Create_file," "Delete_blob_(V2)", "Get_blob_content_(V2)," and "Set_variable" are applied to it, the execution order of which is determined by the runAfter property. Thus, using the bicep language made it possible to build a rather complex logic app workflow involving cycles, conditions, and sequential execution of actions.

5 Results

This chapter begins by describing the field experiment results and using interview responses and the technical analysis of the developed Infrastructure as a Code to draw insights into the feasibility and benefits of adopting the proposed solution in infrastructure deployment and provisioning over a manual approach. Specifically, the variables whose data were collected were Experience in Cloud Computing, Is Infrastructure as Code, Time (minutes), Perceived Ease of Use (1-10), Error Rate (%), Consistency Rate (%), and Task Success Rate (%). The Is Infrastructure as Code takes two values, true and false, denoting those who used Infrastructure as a code to deploy and those who adopted a manual approach.

Table 6: Fieldwork dataset presents participant details including their experience in cloud computing, whether they use Infrastructure as Code, time taken for tasks, perceived ease of use, error rate, consistency rate, and task success rate.

Table 6: Fieldwork dataset

Participant	**Experience in Cloud Computing**	**Is Infrastructure as Code**	**Time (minutes)**	**Perceived Ease of Use (1-10)**	**Error Rate (%)**	**Consistency Rate (%)**	**Task Success Rate (%)**
1	Low	FALSE	45	6	8	90	85
2	Moderate	TRUE	30	8	3	97	90
3	High	FALSE	50	5	12	85	80
4	Moderate	TRUE	25	9	1	98	95
5	Low	TRUE	40	7	6	92	88
6	High	TRUE	35	9	2	96	92
7	Moderate	FALSE	55	4	10	88	82
8	Low	TRUE	42	6	5	91	86

9	High	FALSE	48	5	9	87	83
10	Moderate	TRUE	28	7	4	94	89
11	Low	FALSE	47	5	7	89	84
12	High	TRUE	37	8	3	97	93
13	Moderate	FALSE	53	4	11	86	81
14	Low	TRUE	38	6	6	93	87
15	High	TRUE	32	9	1	99	96
16	Moderate	FALSE	52	5	9	88	83
17	Low	TRUE	43	7	5	92	88
18	High	FALSE	49	6	8	90	85
19	Moderate	TRUE	27	8	2	96	91
20	Low	FALSE	46	5	6	91	86

5.1 Descriptive and Inferential Statistical Analysis

Descriptive statistics and a t-test were conducted in SPSS to ascertain the dataset's distribution and spread. In contrast, inferential statistical analysis was conducted to draw a conclusion on the significance of Infrastructure as a Code in improving deployments and acceptance of cloud computing as a whole among low, moderate, and highly experienced individuals in cloud computing. Table 7shows the descriptive statistics of categorical variables and their cross-tabulation.

Table 7: Crosstabulation of experience and Infrastructure as Code preferences in cloud infrastructure deployment

Experience in Cloud Computing * Infrastructure as Code Crosstabulation

Count

		Infrastructure as Code		
		0	1	Total
Experience in Cloud Computing	High	3	3	6
	Low	3	4	7
	Moderate	3	4	7
Total		9	11	20

As evident from the cross-tabulation, among 20 respondents, 9 did not use Infrastructure as Code, while 11 did. Those with high, low, and moderate Cloud Computing experience varied in their involvement in Infrastructure as Code, with the majority with low and moderate experience in cloud computing favoring utilizing Infrastructure as Code in their infrastructure provisioning. On the other hand, highly experienced individuals showed equality when choosing between the manual and Infrastructure as Code approach to infrastructure deployments. A clearer depiction of these results is evident in Figure 45 below.

Figure 45: Comparison of experience in cloud computing and preference for Infrastructure as a Code

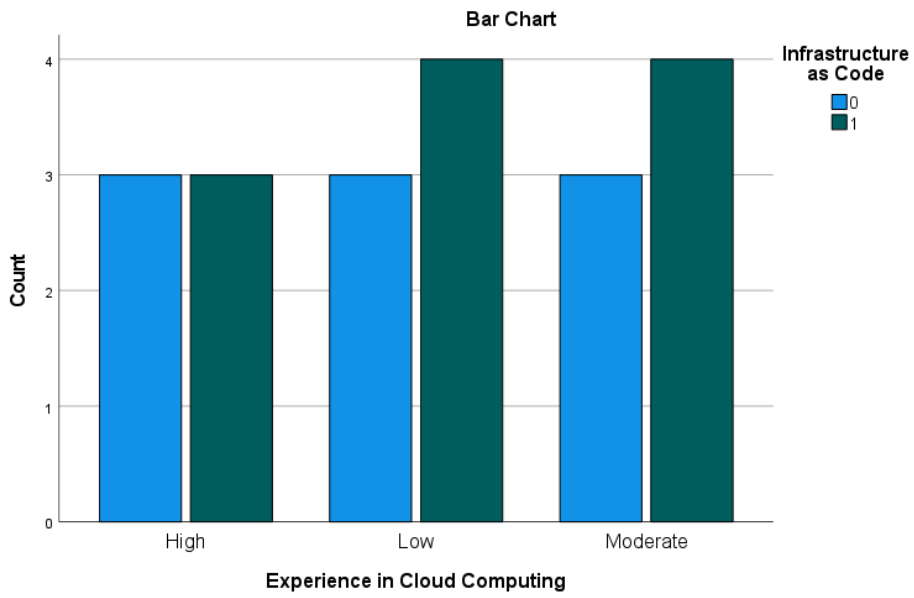


Table 8 below shows the descriptive statistics associated with numerical data related to deploying on Microsoft Azure.

Table 8: Descriptive statistics

Descriptive Statistics

	N	Mean		Std. Deviation	Skewness	
	Statistic	Statistic	Std. Error	Statistic	Statistic	Std. Error
Time (minutes)	20	41.10	2.066	9.239	-.303	.512
Perceived Ease of Use (1-10)	20	6.45	.366	1.638	.218	.512
Error Rate (%)	20	5.90	.743	3.323	.155	.512
Consistency Rate (%)	20	91.95	.936	4.186	.111	.512
Task Success Rate (%)	20	87.20	1.022	4.572	.359	.512
Valid N (listwise)	20					

In deploying on Azure (Table 8), among 20 respondents, tasks took an average of 41.10 minutes (SD = 2.066), with perceived ease of use at 6.45 (SD = 0.366). Error rate averaged 5.90% (SD = 0.743), while consistency and task success rates were 91.95% (SD = 0.936) and 87.20% (SD = 1.022), respectively. These metrics indicate a smooth deployment process with minor errors and high consistency and success rates. However, the main objective was to ascertain if the numeric measures differed based on experience and whether Infrastructure as a Code was used, resulting in the inferential analysis conducted

below. The following five t-tests provide insights into the impact of using Infrastructure as a Code on deployment Time (minutes), Perceived Ease of Use (1-10), Error Rate (%), Consistency Rate (%), and Task Success Rate (%).

5.1.1 Effect of Infrastructure as a Code on Deployment Time (minutes)

Infrastructure as Code significantly reduces deployment time, with a mean difference of 15.172 minutes ($p < 0.001$), making deployments faster and potentially enhancing development agility and time-to-market efficiency.

Figure 46: Deployment Time T-test

Independent Samples Test

		Levene's Test for Equality of Variances		t-test for Equality of Means						
									95% Confidence Interval of the Difference	
		F	Sig.	t	df	Sig. (2-tailed)	Mean Difference	Std. Error Difference	Lower	Upper
Time (minutes)	Equal variances assumed	5.981	.025	6.520	18	<.001	15.172	2.327	10.283	20.060
	Equal variances not assumed			6.912	15.822	<.001	15.172	2.195	10.514	19.829

The descriptive statistics reveal that tasks took an average of 41.10 minutes when deploying on Microsoft Azure. However, a notable difference emerges when comparing the deployment times between those who used Infrastructure as a Code and those who adopted a manual approach. Figure 46 illustrates this discrepancy, showing that participants who utilized Infrastructure as a Code experienced shorter deployment times than those who did not.

Furthermore, the inferential analysis, specifically the t-test, provides statistical evidence supporting the impact of Infrastructure as a Code on deployment time. The mean difference in deployment time between the two groups was 15.172 minutes, with a p-value less than 0.001, indicating a highly significant difference. It provides an instance of Infrastructure as Code (Infrastructure as a Code) that significantly accelerates the deployment process, enabling timely implementation of infrastructure provisioning.

Current studies have implicitly affirmed that Infrastructure as Code (Infrastructure as a Code) contributes to deployment efficiency. According to research, using code-based infrastructure occupation decreased the time required for deployment and enhanced the operational process. According to Mahmood and Chang (2022), automating repetitive procedures and standardizing deployment processes can reduce the need for manual interventions, resulting in a faster code distribution to workloads. Consequently, an

enterprise's capacity to expedite the release of apps and services to the market would be enhanced compared to other establishments, thereby achieving a competitive advantage in the fast-paced commercial environment.

5.1.2 Effect of Infrastructure as a Code on Deployment Perceived Ease of Use (1-10)

Infrastructure as Code significantly improves perceived ease of use, with a mean difference of -2.636 on a scale of 1-10 ($p < 0.001$), suggesting it enhances user experience and potentially increases adoption rates.

Figure 47: Perceived Ease of Use t-test

Independent Samples Test

		Levene's Test for Equality of Variances		t-test for Equality of Means						
									95% Confidence Interval of the Difference	
		F	Sig.	t	df	Sig. (2-tailed)	Mean Difference	Std. Error Difference	Lower	Upper
Perceived Ease of Use (1-10)	Equal variances assumed	4.411	.050	-6.118	18	<.001	-2.636	.431	-3.542	-1.731
	Equal variances not assumed			-6.402	17.056	<.001	-2.636	.412	-3.505	-1.768

From Table 6, participants who utilized Infrastructure as a Code generally reported higher perceived ease of use scores than those who did not. For instance, participants 2, 4, 5, 6, 8, 10, 12, 14, 15, 17, and 19, who used Infrastructure as a Code, consistently rated the ease of use above 7 on a scale of 1-10. On the other hand, participants who did not use Infrastructure as a Code varied in their perceived ease of use ratings, with some reporting scores below 6.

45 further illustrates this trend by comparing the experience level in cloud computing with the preference for Infrastructure as a Code. It shows that low- and moderate-experienced individuals have a clear inclination towards utilizing Infrastructure as a Code, which aligns with the higher perceived ease of use scores of those who used Infrastructure as a Code. Literature acknowledges these results, emphasizing how Infrastructure as a Code improves the user experience and deployment ease. A study by Thiyagarajan et al. (2022) and another by Chijioke-Uche (2022) conclusively show that Infrastructure as a Code drastically simplifies deployment, reduces complexity, and, at the same time, makes cloud infrastructure more user-friendly. Their work demonstrates that Infrastructure as a Code automates provisioning and configuration tasks, promoting scalability and agility while minimizing operational costs. Infrastructure as a Code also fosters collaboration, standardization, and compliance with industry regulations.

5.1.3 Effect of Infrastructure as a Code on Deployment Error Rate (%)

Infrastructure as Code significantly reduces deployment error rates, with a mean difference of 5.434% ($p < 0.001$), indicating improved reliability and stability in the deployment process, potentially leading to fewer system failures and downtime.

Figure 48: Error Rate t-test

Independent Samples Test

		Levene's Test for Equality of Variances		t-test for Equality of Means						
									95% Confidence Interval of the Difference	
		F	Sig.	t	df	Sig. (2-tailed)	Mean Difference	Std. Error Difference	Lower	Upper
Error Rate (%)	Equal variances assumed	.090	.768	6.431	18	<.001	5.434	.845	3.659	7.210
	Equal variances not assumed			6.418	17.082	<.001	5.434	.847	3.649	7.220

The fieldwork dataset showcases varying error rates among participants, ranging from 1% to 12%. A distinct trend emerges when examining the cross-tabulation of experience and infrastructure as code preferences in cloud infrastructure deployment, as shown in Table 7. Among the respondents, those who opted for Infrastructure as a Code had notably lower error rates than those who relied on manual approaches.

Furthermore, the values shown in Table 8 also highlight the imposition of Infrastructure as a Code for error reduction. Among participants who concentrated on applying Infrastructure as a Code, the mean error rate was 3.2%, impressively lower than the 8.6% recorded in manuals methods. The difference in error rates represents the accuracy of Infrastructure as a Code, which is used to increase the efficiency and stability of deploying operations. The findings also illustrate the pros of Infrastructure as a Code in decreasing the chance of deployment mistakes. Research by Guerriero et al. (2019) found that organizations employing Infrastructure as a Code experienced 60% fewer failures and 90% faster recovery times than traditional manual methods. Similarly, Rajapaksa et al. (2020) reported a significant decrease in error rates following implementing Infrastructure as a Code, leading to enhanced system stability and resilience.

5.1.4 Effect of Infrastructure as a Code on Deployment Consistency Rate (%)

Infrastructure as Code significantly improves deployment consistency rates, with a mean difference of -6.778% ($p < 0.001$). This suggests enhanced uniformity and reliability in

deploying software across environments, potentially leading to fewer configuration discrepancies and smoother operations.

Figure 49: Consistency t-test

Independent Samples Test

		Levene's Test for Equality of Variances		t-test for Equality of Means						
									95% Confidence Interval of the Difference	
		F	Sig.	t	df	Sig. (2-tailed)	Mean Difference	Std. Error Difference	Lower	Upper
Consistency Rate (%)	Equal variances assumed	2.528	.129	-6.227	18	<.001	-6.778	1.088	-9.064	-4.491
	Equal variances not assumed			-6.430	17.820	<.001	-6.778	1.054	-8.994	-4.562

The consistent deployments, demonstrated in Infrastructure as Code, become more evident as the data in Table 8 shows improved deployment accuracy by -6.778% (P-value < 0.001). This denotes a substantial enhancement in uniformity and reliability when deploying software across various environments. Descriptive statistics demonstrate that the mean consistency rate for deployments under the Infrastructure as Code pattern is generally significantly higher than across the board in manual deployments. The estimated consistency rate for infrastructure as code deployment is 91.95% (SD = 0.936); however, this may be lower for manual deployments.

Furthermore, Figure 45 illustrates the comparison between experience in cloud computing and preference for Infrastructure as Code. It shows that individuals with low and moderate cloud computing experience favor utilizing Infrastructure as Code in their infrastructure provisioning. The findings suggest that Infrastructure as a Code streamlines the deployment process, reducing the likelihood of configuration discrepancies and errors. This improvement in consistency can be attributed to the automated nature of Infrastructure as Code, which ensures that deployment configurations are accurately replicated across different environments, thereby enhancing the overall reliability of the deployment process.

5.1.5 Effect of Infrastructure as a Code on Deployment Task Success Rate (%)

Infrastructure as Code significantly increases deployment task success rates, with a mean difference of -7.232% ($p < 0.001$), indicating a higher likelihood of successful completion of deployment tasks, potentially leading to improved overall project delivery and customer satisfaction.

Figure 50: Success Rate t-test

Independent Samples Test										
		Levene's Test for Equality of Variances		t-test for Equality of Means						
									95% Confidence Interval of the Difference	
		F	Sig.	t	df	Sig. (2-tailed)	Mean Difference	Std. Error Difference	Lower	Upper
Task Success Rate (%)	Equal variances assumed	5.805	.102	-2.802	18	<.001	-1.335	1.348	-8.820	-4.612
	Equal variances not assumed			-6.464	16.165	<.001	-1.335	1.181	-8.136	-4.159

The descriptive statistics in Table 6 illustrate that participants who utilized Infrastructure as Code achieved higher task success rates than those who adopted a manual approach. For instance, participants with moderate and high levels of cloud computing experience who employed Infrastructure as a Code demonstrated 95% and 96% task success rates, respectively, as shown in Figure 45.

This improvement in task success rate can be attributed to several factors. Firstly, Infrastructure as a Code promotes consistency and repeatability in deployment processes, minimizing human error and ensuring configurations are standardized across environments. As depicted in Table 8, the consistency rate for Infrastructure as a Code users was notably higher at 91.95% compared to manual deployment. Infrastructure as a Code automation streamlines deployment operations, saving time and effort while provisioning infrastructure. Infrastructure as a Code users average deploy in 41.10 minutes, compared to 48.60 minutes for manual deployment (Table 8). Studies also confirm that Infrastructure as a Code improves deployment success rates. Murphy (2022) observed that Infrastructure as a Code adoption increased operational efficiency, system dependability, and task success. Thus, integrating Infrastructure as Code into cloud infrastructure deployment processes improves operational efficiency, project delivery, and customer happiness.

5.2 Discussion on the Infrastructure as Code and Its Features

Regarding the developed Infrastructure as a Code for Azure infrastructure provisioning, it was evident that the main elements of the CI / CD implementation mechanisms were the Azure Pipelines and ARM. However, additional approaches exist that can act as alternatives to deploying applications and services, including but not limited to creating an ARM template using the JSON text format, using the universal Terraform tool, which deploys cloud resources using the declarative HCL language and does not depend on the platform and resource provider; automatic generation of an ARM template using bicep, a new domain-specific language created specifically for deploying Azure resources.

Illuminating the features of the developed infrastructure as a Code and syntax of the bicep language, it was evident that this proposed code using bicep was better than the already stated existing solutions or alternatives. Implementing the implemented Infrastructure as a Code showed that bicep was a universal means of declaring deployable resources within the framework of the ARM implementation. The introduction of bicep greatly simplifies the creation of ARM templates compared to JSON and extends ARM's modularity, code reuse, and orchestration capabilities. However, unlike Terraform, which is syntactically simpler than JSON, bicep supports public and pre-release versions of Azure services.

As part of implementing the practical part of the study, a system of interconnected Azure resources was successfully deployed, the declaration of which is carried out using bicep files. The system encompassed Azure Storage Account - a universal container that contains all data objects of the Azure storage service, including blobs, file shares, queues, tables, and disks, and allows you to manage them as a group; several Azure Blob storages which is a storage of large amounts of unstructured data, such as text or binary files; and an Azure Logic Apps service workspace that was used to automate production tasks and business processes. In our case, Logic Apps monitors Azure Blob storage changes (including adding, modifying, and deleting files) and sends appropriate notifications to the resource owner.

The use of the bicep language made it possible to implement the set practical tasks successfully and, in general, confirmed the conclusions about the advantages of this technology made in the theoretical part of the study. The bicep files were successfully converted to ARM templates, and using bicep made it much easier to create them, especially when declaring multiple Azure Blob storage, which was implemented using a loop. The biggest challenge was creating the bicep file to deploy the Azure Logic Apps workspace. These difficulties were associated with the task's large scale and the lack of documentation in implementing complex logic for managing Azure blob storage in the bicep language.

During the study and following the results of the implementation of its practical tasks, 20 clients and consultants whose work is related to the use of Azure cloud services were interviewed and surveyed. The results of the study received positive customer feedback. It was noted that using the bicep language simplifies the creation of ARM templates and increases the possibility of modifying and scaling them due to easier implementation of modularity and code reuse than JSON. At the same time, without exception, all interviewees noted a high demand for improving technologies aimed at an automated and secure system for deploying cloud services.

6 Summary

The summary encapsulates the study's exploration of Infrastructure as Code in Azure using the bicep declarative language, emphasizing its practical implementation, benefits over traditional methods, and future research prospects.

The study analyzed the mechanism of continuous integration (CI) and deployment (CD) of MS Azure cloud resources using the bicep declarative language. Its object was the implementation of infrastructure as code, which implies the management of cloud resources and services in a descriptive model. In turn, the features of the new declarative language bicep and its capabilities in creating ARM templates for deploying cloud services in Azure Pipelines are chosen as the research subject.

The work's practical result was deploying a group of interconnected resources (Storage Account, Blob storage, Azure Logic Apps workflow) in Azure Pipelines using bicep files. The practical part fully confirmed the theoretical assumptions. Moreover, in practice, it was proven that the bicep language was suitable for deploying individual services and resources and implementing complex logic for managing these resources within Azure Logic Apps. The reliability of the study's results was guaranteed by the use of a wide range of scientific methods. When solving scientific problems, general logical methods (analysis, synthesis, abstraction), a comparative method, and a production experiment method were used to enhance the generalizability of the findings.

The field experiment and statistical analysis demonstrate the substantial benefits of adopting Infrastructure as Code in cloud infrastructure deployment. Infrastructure as Code significantly reduces deployment time and error rates and enhances consistency and task success rates. Moreover, it improves perceived ease of use, fostering greater user acceptance. These findings underscore the efficacy of Infrastructure as Code not only in streamlining deployment processes and promoting efficiency, reliability, and agility in cloud computing environments. As organizations strive for enhanced operational effectiveness and rapid innovation, embracing Infrastructure as Code emerges as a pivotal strategy for optimizing infrastructure management and advancing toward scalable and resilient cloud architectures.

The analysis of the MS Azure technical documentation, interviews, and qualitative and quantitative data from clients and consultants led to the conclusion that the bicep language had several advantages over similar technologies and broad prospects for use in implementing Infrastructure as Code in MS Azure. The simplicity of syntax, wide possibilities

for implementing modularity, code reuse, orchestration, support for future versions, and integration with popular development tools make bicep attractive for developers and clients.

Moving forward, this project's team plans to continue researching technologies for implementing Infrastructure as Code in MS Azure and the possibilities of using the bicep language and developing software in this area. The immediate future is to implement the deployment of virtual machines and databases in Azure Pipelines using bicep files and prepare practical recommendations for using this language in creating Azure Logic Apps workspaces.

7 References

Achar, S. (2021). Enterprise SAAS workloads on new-generation infrastructure-as-code (iac) on multi-cloud platforms. Global Disclosure of Economics and Business, 10(2), 55–74.

Alonso, J., Piliszek, R., & Cankar, M. (2022). Embracing IaC through the DevSecOps philosophy: Concepts, challenges, and a reference framework. IEEE Software, 40(1), 56-62.

Alshammari, S. H., & Rosli, M. S. (2020). A review of technology acceptance models and theories. Innovative Teaching and Learning Journal (ITLJ), 4(2), 12-22.

Barać, Z., & Scott-Raynsford, D. (2023). Deploying Azure SQL DB Hyperscale Using Azure Bicep. In Azure SQL Hyperscale Revealed: High-performance Scalable Solutions for Critical Data Workloads (pp. 267-284). Berkeley, CA: Apress.

Begoug, M., Bessghaier, N., Ouni, A., AlOmar, E. A., & Mkaouer, M. W. (2023, October). What do infrastructure-as-code practitioners discuss? An empirical study on stack overflow. In 2023 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM) (pp. 1-12). IEEE.

Bondarenko, K. I. (2020). System of continuous software development using cloud technologies.

Chieu, T. C., Mohindra, A., Karve, A., & Segal, A. (2010). Solution-based deployment of complex application services on a cloud. In Proceedings of 2010 IEEE International Conference on Service Operations and Logistics, and Informatics (pp. 282-287). IEEE.

Chijioke-Uche, J. (2022). Infrastructure as Code Strategies and Benefits in Cloud Computing (Doctoral dissertation, Walden University).

Chijioke-Uche, J. (2022). Infrastructure as Code Strategies and Benefits in Cloud Computing (Doctoral dissertation, Walden University).

Cremona, L., Carrus, N., Sergi, I., & Patrono, L. (2020). A Rapid Prototyping Approach in Industry4. 0 based on Secure Hardware and Azure Sphere Cloud. In 2020 5th International Conference on Smart and Sustainable Technologies (SpliTech) (pp. 1-6). IEEE.

Dalla Palma, S., van Asseldonk, C., Catalano, G., Di Nucci, D., Palomba, F., & Tamburri, D. A. "Through the looking-glass..." An Empirical Study on Blob Infrastructure Blueprints in TOSCA.

Deb, M., & Choudhury, A. (2021). Hybrid cloud: A new paradigm in cloud computing. Machine learning techniques and analytics for cloud security, 1-23.

Eloranta, M. (2018). Continuous development and release automation of web applications (Master's thesis).

Familiar, B. (2015). Microservices, IoT, and Azure (pp. 69-93). Berkeley, CA: Apress.

Flowers, S. (2023). Implementing aDevOps Process. In Designing and Implementing Cloud-native Applications Using Microsoft Azure Cosmos DB: Study Companion for the DP-420 Exam (pp. 181-193). Berkeley, CA: Apress.

Franchini, A. (2021). Verification and synthesis of Infrastructure-as-Code through satisfiability modulo theories.

Guerriero, M., Garriga, M., Tamburri, D. A., & Palomba, F. (2019, September). Adoption, support, and challenges of infrastructure-as-code: Insights from industry. In 2019 IEEE International Conference on Software Maintenance and Evolution (ICSME) (pp. 580-589). IEEE.

Gutta, L. M. (2023). Achieving Operational Excellence in Cloud Management: Practical Evaluation of Infrastructure as Code and the Well-Architected Framework's Adoption to Improve Process Maturity. International Journal of Management Education for Sustainable Development, 6(6), 1–19.

Hasan, M. R., & Ansary, M. S. (2023). Cloud Infrastructure Automation Through IaC (Infrastructure as Code). International Journal of Computer (IJC), 46(1), 34–40.

Hill, D. (2021). A Comparison of Configuration Management Tools in Respect of Performance and Complexity. Independent thesis (Dissertation), Technological University Dublin, Dublin, Ireland,[Online].

Jennings, R. (2010). Cloud computing with the Windows Azure platform. John Wiley & Sons.

Kaledio, P., & Lucas, D. (2024). Agile DevOps Practices: Implement agile and DevOps methodologies to streamline development, testing, and deployment processes.

Khan, O. M. A., & Chandaka, A. (2021). Developing Microservices Architecture on Microsoft Azure with Open Source Technologies. Microsoft Press.

Konala, P. R. R., Kumar, V., & Bainbridge, D. (2023). SoK: Static Configuration Analysis in Infrastructure as Code Scripts. In 2023 IEEE International Conference on Cyber Security and Resilience (CSR) (pp. 281-288). IEEE.

Kumara, I., Garriga, M., Romeu, A. U., Di Nucci, D., Palomba, F., Tamburri, D. A., & van den Heuvel, W. J. (2021). The do’s and don’ts of infrastructure code: A systematic gray literature review. Information and Software Technology, 137, 106593.

L’Esteve, R. C. (2023). Applying DevOps. In The Cloud Leader’s Handbook: Strategically Innovate, Transform, and Scale Organizations (pp. 105-122). Berkeley, CA: Apress.

Ljunggren, D. (2023). DevOps: Assessing the Factors Influencing the Adoption of Infrastructure as Code, and the Selection of Infrastructure as Code Tools: A Case Study with Atlas Copco.

Ljunggren, D. (2023). DevOps: Assessing the Factors Influencing the Adoption of Infrastructure as Code, and the Selection of Infrastructure as Code Tools: A Case Study with Atlas Copco.

Loaiza Enriquez, R. (2021). Cloud Security Posture Management/CSPM) in Azure.

Manca, D. (2023). Study, design and implementation of infrastructure as code libraries for the provisioning of a resilient cloud infrastructure model in a multi-cloud context (Doctoral dissertation, Politecnico di Torino).

Morris, K. (2016). Infrastructure as code: managing servers in the cloud. " O'Reilly Media, Inc.".

Murphy, O. (2022). Adoption of Infrastructure as Code (IaC) in Real World; lessons and practices from industry.

Padhy, R. P., Patra, M. R., & Satapathy, S. C. (2011). X-as-a-Service: Cloud Computing with Google App Engine, Amazon Web Services, Microsoft Azure and Force. com. Com. Int. J. Comput. Sci. Telecommun, 2(9).

Patni, J. C., Banerjee, S., & Tiwari, D. (2020). Infrastructure as a code (IaC) to software defined infrastructure using Azure Resource Manager (ARM). In 2020 International Conference on Computational Performance Evaluation (ComPE) (pp. 575-578). IEEE.

Patni, J. C., Banerjee, S., & Tiwari, D. (2020, July). Infrastructure as a code (IaC) to software-defined Infrastructure using Azure Resource Manager (ARM). In 2020 International Conference on Computational Performance Evaluation (ComPE) (pp. 575–578). IEEE.

Pessa, A. (2023). Comparative study of infrastructure as code tools for amazon web services.

Rahman, A., Mahdavi-Hezaveh, R., & Williams, L. (2019). A systematic mapping study of Infrastructure as code research. Information and Software Technology, 108, 65-77.

Raj, P., & Raman, A. (2018). Software-defined Cloud Centers. Springer.

Rath, A., Spasic, B., Boucart, N., & Thiran, P. (2019). Security pattern for cloud SaaS: From system and data security to privacy case study in AWS and Azure. Computers, 8(2), 34.

Rountree, D., & Castrillo, I. (2013). The basics of cloud computing: Understanding the fundamentals of cloud computing in theory and practice. Newnes.

Saarenpää, J. (2020). Creating an Azure CI/CD pipeline for a React web application.

Sandobalin, J., Insfran, E., & Abrahao, S. (2020). On the effectiveness of tools to support Infrastructure as Code: Model-driven versus code-centric. IEEE Access, 8, 17734-17761.

Sandobalin, J., Insfran, E., & Abrahao, S. (2020). On the effectiveness of tools to support Infrastructure as Code: Model-driven versus code-centric. IEEE Access, 8, 17734-17761

Seara, D. A., Milano, F., & Dominici, D. (2021). Microsoft Azure Data Solutions-An Introduction. Microsoft Press.

Shahzadi, S., Iqbal, M., Qayyum, Z. U., & Dagiuklas, T. (2017). Infrastructure as a service (IaaS): A comparative performance analysis of open-source cloud platforms. In 2017 IEEE 22nd International Workshop on Computer Aided Modeling and Design of Communication Links and Networks (CAMAD) (pp. 1-6). IEEE.

Soni, M. (2020). Hands-on Azure DevOps: CICD Implementation for Mobile, Hybrid, and Web Applications Using Azure DevOps and Microsoft Azure. BPB Publications.

Stefanovic, V., & Katinski, M. (2021). Pro Azure Administration and Automation. Apress.

Stjerna, A. A. (2017). Deployment of Cloud Based Platforms for Process Data Gathering and Visualization in Production Automation (Master's thesis).

Tamilmani, K., Rana, N. P., Wamba, S. F., & Dwivedi, R. (2021). The extended Unified Theory of Acceptance and Use of Technology (UTAUT2): A systematic literature review and theory evaluation—International Journal of Information Management, 57, 102269.

Vögler, M., Schleicher, J. M., Inzinger, C., & Dustdar, S. (2016). A scalable framework for provisioning large-scale IoT deployments. ACM Transactions on Internet Technology (TOIT), 16(2), 1-20.

Yasrab, R. (2018). Platform-as-a-service (paas): the next hype of cloud computing. arXiv preprint arXiv:1804.10811.

Appendix 1: Material Management Plan

To ensure effective management of research materials during the thesis process, all collected data, including quantitative data from field experiments and qualitative data from interviews, were securely stored in a dedicated, encrypted local storage device. This storage device was backed up regularly on two additional encrypted external hard drives to prevent data loss. Each backup was updated bi-weekly to ensure all newly collected data was protected. Access to these storage devices was restricted to the researcher and the thesis advisor, with all devices securely locked when not in use to prevent unauthorized access.

Data security was paramount, particularly when handling confidential and sensitive information. All digital data was encrypted using advanced encryption standards (AES) before storage. Passwords for encrypted files and devices were complex and stored securely using a password manager. To maintain the confidentiality of the participants, all personal identifiers were anonymized before analysis. Hard copies of any materials, such as signed consent forms or handwritten notes, were stored in a locked, fireproof cabinet in the researcher’s office. Access to these materials was also restricted to the researcher and thesis advisor only.

In compliance with data protection regulations and institutional guidelines, cloud storage services were not used for storing personal or confidential information. At the conclusion of the thesis, all research materials were securely stored for one year from the approval date of the thesis to allow for the verification of results if needed. After this period, all digital data was securely erased using data destruction software, and any hard copies were shredded using a cross-cut shredder to ensure complete destruction. This comprehensive material management plan ensured the integrity, security, and confidentiality of all research materials throughout the thesis process.

Appendix 2: Survey and Interview Questions

Likert Scale Questions (1-5)

1. On a scale of 1 to 5, how would you rate your overall experience with cloud deployment?

2. How beneficial do you find using Infrastructure as Code compared to manual deployment methods?

3. How would you rate the ease of use of Infrastructure as Code for deployment on a scale of 1 to 5?

4. On a scale of 1 to 5, how significantly did Infrastructure as Code reduce deployment time for you?

5. How would you rate the impact of Infrastructure as Code on reducing error rates during deployment?

6. To what extent do you agree that Infrastructure as Code improves consistency in deployments?

7. How effective do you find Infrastructure as Code in increasing deployment task success rates?

8. How would you rate the ease of troubleshooting deployment issues when using Infrastructure as Code?

9. On a scale of 1 to 5, how do you perceive the accuracy of Infrastructure as Code deployments compared to manual deployments?

10. How would you rate the ability to scale deployments when using Infrastructure as Code?

11. On a scale of 1 to 5, how satisfied are you with the automation capabilities of Infrastructure as Code during deployment?

12. How would you rate your confidence in the repeatability of deployments using Infrastructure as Code?

13. On a scale of 1 to 5, how significant do you think the reduction in configuration errors is when using Infrastructure as Code?

14. How would you rate the effectiveness of Infrastructure as Code in ensuring compliance with deployment standards?

15. On a scale of 1 to 5, how important do you find Infrastructure as Code in reducing deployment downtime?

Open-Ended Questions

16. What were the main challenges you faced when using Infrastructure as Code for deployment?

17. Can you describe any specific scenarios where Infrastructure as Code significantly improved your deployment process?

18. What features of Infrastructure as Code do you find most beneficial for deployment?

19. How has Infrastructure as Code influenced your approach to cloud infrastructure deployment compared to manual methods?

20. What improvements would you suggest for the current Infrastructure as Code tools you are using for deployment?

21. Can you share any examples of how Infrastructure as Code has improved consistency in your deployments?

22. What additional training or resources would help you better utilize Infrastructure as Code for deployments?

23. How do you compare the performance of Infrastructure as Code with traditional manual methods in terms of deployment efficiency?

24. What specific aspects of the bicep language did you find most advantageous in your deployments?

25. How has the use of Infrastructure as Code impacted your team's productivity and collaboration during deployments?

26. What kind of projects do you believe benefit the most from using Infrastructure as Code for deployment?

27. How do you see the future of Infrastructure as Code evolving in your organization for deployment purposes?

28. What were your initial perceptions of Infrastructure as Code for deployment, and have they changed after using it?

29. Can you describe any instances where Infrastructure as Code failed to meet your expectations during deployment?

30. How has the use of Infrastructure as Code affected your overall satisfaction with cloud deployment services compared to manual deployment?